

Universidad Internacional de la Rioja (UNIR)

**Escuela Superior de Ingeniería y
Tecnología**

Master universitario en Inteligencia Artificial

Diseño de un asistente institucional, RAG e integración de herramientas

Trabajo Fin de Máster

Presentado por: Nuria Iglesias Traviesa

Director: David Jiménez Cabello

Ciudad: Valencia

Fecha: 22/04/2026

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Planteamiento del trabajo	2
1.3. Estructura del trabajo	4
2. Contexto y estado del arte	6
2.1. Contexto	6
2.2. Estado del arte	7
2.2.1. Agentes y orquestación	8
2.2.2. Uso de herramientas e integración de capacidades	8
2.2.3. RAG	9
2.2.4. Evaluación y mejora concreta del retrieval	10
2.2.5. Asistentes institucionales	11
2.3. Encaje de la propuesta	12
2.4. Conclusiones	13
3. Objetivos y Metodología	15
3.1. Objetivos Generales	15
3.2. Objetivos Específicos	15
3.3. Criterios de comprobación de los objetivos	16
3.4. Metodología del trabajo	17
3.4.1. Scrum	18
3.4.2. Roles	18
3.4.3. Artefactos	18
3.4.4. Eventos	19
3.4.5. Adaptación al proyecto	19
3.5. Riesgos y planes de contingencia	19

3.6. Planificación del trabajo	21
3.6.1. Sprint 1: análisis y definición del problema	21
3.6.2. Sprint 2: diseño de la arquitectura y preparación del entorno de trabajo	21
3.6.3. Sprint 3: implementación del <i>baseline</i> del asistente	22
3.6.4. Sprint 4: integración de la funcionalidad accionable	22
3.6.5. Sprint 5: implementación de la mejora del componente de recuperación de información	22
3.6.6. Sprint 6: diseño de la evaluación y comparación experimental	23
3.6.7. Sprint 7: Revisión final, análisis de resultados y redacción de la memoria	23
3.7. Recursos y costes	23
3.7.1. Recursos humanos	24
3.7.2. Recursos técnicos	24
3.7.3. Estimación de dedicación	24
3.7.4. Estimación de costes	25
4. Diseño e implementación de la propuesta	26
4.1. Arquitectura general del sistema	26
4.2. Flujo de ejecución del asistente	27
4.3. Papel del orquestador	28
4.4. Herramientas disponibles	29
4.5. Variantes implementadas	29
4.6. Lenguaje y entorno de desarrollo y ejecución	30
4.6.1. Python	30
4.6.2. Docker	31
4.7. Procesamiento documental	32
4.7.1. Obtención del corpus y scraping web	32
4.7.2. Conversión y normalización de documentos	34
4.7.3. Segmentación del corpus y metadatos	34
4.8. Almacenamiento vectorial	35
4.9. Modelos de representación semántica	36
4.9.1. BGE-M3	36
4.10. Recuperación de información	36
4.10.1. Recuperación semántica	37
4.10.2. Recuperación híbrida con BM25	37
4.11. Modelo de lenguaje	37

4.12. Mejora de la recuperación mediante reranking	38
4.12.1. Modelo de reranking seleccionado	38
4.13. Interfaz de usuario y observabilidad	38
4.13.1. Interfaz conversacional con Streamlit	38
4.13.2. Trazabilidad y monitorización con Phoenix	39
5. Resultados experimentales	40
5.1. Diseño de la evaluación	40
5.1.1. Corpus y dataset de evaluación	40
5.1.2. Métricas empleadas	42
5.1.3. Variantes evaluadas	43
5.2. Resultados de recuperación	43
5.3. Resultados de calidad de respuesta	44
5.4. Evaluación funcional del orquestador	45
5.5. Discusión de resultados	46
5.6. Limitaciones de la evaluación	47
5.7. Síntesis de resultados	47
Bibliography	48
Appendices	53
Apéndice A. Fusce viverra lectus	54

Resumen

Este Trabajo de Fin de Máster presenta el diseño, implementación y evaluación de un asistente institucional basado en Retrieval-Augmented Generation (RAG) e integración de herramientas. La propuesta no se plantea como el desarrollo de un chatbot convencional, sino como una arquitectura modular, trazable y supervisada capaz de combinar recuperación documental, mejora del ranking de evidencias, orquestación del flujo de interacción y generación controlada de borradores de correo electrónico.

El sistema se orienta a un contexto institucional universitario, donde la información relevante para el usuario puede encontrarse dispersa en páginas web, normativas, documentos administrativos y guías de procedimiento. Para abordar este escenario, se construye un corpus documental a partir de fuentes públicas, se normaliza su contenido, se segmenta en fragmentos recuperables y se indexa en una base documental preparada para búsquedas semánticas e híbridas. Sobre esta base, el asistente permite responder consultas en lenguaje natural utilizando evidencia recuperada y manteniendo trazabilidad sobre las fuentes empleadas.

Desde el punto de vista experimental, el trabajo compara distintas variantes del sistema con el objetivo de analizar el impacto de los componentes incorporados. En primer lugar, se establece un baseline basado en recuperación semántica. Posteriormente, se evalúa una variante mejorada mediante recuperación híbrida y reranking, orientada a mejorar la posición de las evidencias relevantes. Finalmente, se analiza el papel del orquestador como mecanismo de control del flujo, abstención ante preguntas no respondibles y activación de herramientas accionables.

Los resultados muestran que la recuperación híbrida con reranking mejora las métricas de recuperación respecto al baseline, especialmente en la posición de la fuente esperada dentro del ranking. Por su parte, el orquestador aporta valor principalmente en términos de control, trazabilidad, abstención ante ausencia de evidencia y generación supervisada de borradores de correo, aunque no siempre mejora la cobertura factual frente al flujo determinista. En conjunto, el trabajo demuestra la utilidad de una arquitectura modular para asistentes institucionales privados, donde la calidad de la recuperación y la supervisión del flujo cumplen funciones complementarias.

Palabras clave: Modelos de lenguaje de gran tamaño, Retrieval-Augmented Generation, recuperación híbrida, reranking, orquestación, uso de herramientas,

asistente institucional.

Abstract

This Master's Thesis presents the design, implementation and evaluation of an institutional assistant based on Retrieval-Augmented Generation (RAG) and tool integration. The proposal is not conceived as the development of a conventional chatbot, but as a modular, traceable and supervised architecture that combines document retrieval, evidence ranking improvement, interaction flow orchestration and controlled generation of email drafts.

The system is designed for a university institutional context, where relevant information for users may be distributed across web pages, regulations, administrative documents and procedural guidelines. To address this scenario, a document corpus is built from public sources, normalized, divided into retrievable chunks and indexed in a document base prepared for semantic and hybrid search. On top of this base, the assistant answers natural language queries using retrieved evidence while preserving traceability over the sources used.

From an experimental perspective, the work compares different system variants in order to analyse the impact of the incorporated components. First, a baseline based on semantic retrieval is established. Then, an improved variant based on hybrid retrieval and reranking is evaluated, with the aim of improving the position of relevant evidence in the ranking. Finally, the role of the orchestrator is analysed as a mechanism for flow control, abstention in unanswerable questions and activation of actionable tools.

The results show that hybrid retrieval with reranking improves retrieval metrics compared to the baseline, especially regarding the position of the expected source in the ranking. The orchestrator, in turn, provides value mainly in terms of control, traceability, abstention when evidence is insufficient and supervised generation of email drafts, although it does not always improve factual coverage compared to the deterministic flow. Overall, the work demonstrates the usefulness of a modular architecture for private institutional assistants, where retrieval quality and flow supervision play complementary roles.

Keywords: Large Language Models, Retrieval-Augmented Generation, hybrid retrieval, reranking, orchestration, tool use, institutional assistant.

Índice de figuras

1.	Arquitectura general del asistente institucional propuesto.	4
2.	Diagrama de Gantt con la planificación temporal del trabajo.	23

Índice de Tablas

1.	Estimación de horas dedicadas al proyecto.	25
2.	Estimación orientativa de costes humanos del proyecto.	25
3.	Resumen del proceso de construcción del corpus documental.	33
4.	Resumen del corpus y del dataset utilizados en la evaluación.	42
5.	Distribución de preguntas del dataset de evaluación.	42
6.	Variantes evaluadas durante los experimentos.	43
7.	Comparativa de recuperación entre el baseline y la variante con recuperación híbrida y reranking.	44
8.	Comparativa de calidad de respuesta entre el flujo determinista y el orquestador.	44
9.	Validación funcional del comportamiento del orquestador.	45

Capítulo 1

Introducción

En este primer capítulo se presenta el sistema de agentes con integración de herramientas orientadas a la realización de acciones útiles para el usuario, así como la propuesta de mejora del componente de recuperación de información. Asimismo, se describen la motivación y el contexto del trabajo, el planteamiento general de la solución, los objetivos a grandes rasgos y la estructura del documento.

1.1. Motivación

La motivación principal de este trabajo surge de la necesidad de explorar cómo diseñar un asistente institucional que combine consulta documental y capacidad de realizar acciones. Frente a enfoques centrados exclusivamente en la generación de respuestas, se parte de la idea de que un asistente útil en un entorno institucional debe ser capaz no solo de informar, sino también de ayudar a materializar el siguiente paso de manera controlada y supervisada por el ser humano.

Esta necesidad resulta especialmente visible en el ámbito universitario, donde parte de la información relevante para el estudiante se encuentra dispersa en normativas, páginas web, documentos administrativos o guías internas cuya consulta no siempre es sencilla. Cuestiones relacionadas con trámites, prácticas, horarios o becas pueden generar dudas incluso cuando la información existe, simplemente porque no resulta fácilmente accesible o comprensible. En este contexto, disponer de un asistente capaz de responder de forma clara, fundamentada y contextualizada puede suponer una mejora significativa en la experiencia del usuario, especialmente si se tiene en cuenta que muchas instituciones universitarias todavía no cuentan con herramientas conversacionales suficientemente útiles, especializadas o integradas para este tipo de consultas.

A ello se añade una segunda motivación de carácter investigador. En sistemas basados en *Retrieval-Augmented Generation* (RAG), la calidad de la recuperación documental condiciona directamente la calidad de las respuestas generadas. Por este motivo, resulta de interés analizar si una mejora concreta del componente de recuperación, basada en estrategias de recuperación híbrida y *reranking* (Nogueira y col., 2020a), puede traducirse en una mejora observable en las respuestas y en la utilidad general del asistente. De este modo, el trabajo no solo responde a una necesidad práctica de desarrollo, sino también a una pregunta aplicada de investigación sobre el impacto real de optimizar el *retrieval* en asistentes institucionales.

1.2. Planteamiento del trabajo

Se propone el desarrollo de un asistente conversacional institucional privado que permita consultar documentación interna, ofrecer respuestas fundamentadas y apoyar acciones útiles para el usuario dentro de un entorno controlado.

La contribución principal de este Trabajo de Fin de Máster no se limita al desarrollo de un chatbot convencional, sino que se centra en el diseño, implementación y evaluación de una arquitectura modular, trazable y supervisada para un asistente institucional. La propuesta combina de forma integrada cuatro capacidades complementarias: la recuperación documental sobre un corpus institucional, la mejora del ranking de evidencias mediante recuperación híbrida y *reranking*, la orquestación del flujo de interacción y la generación controlada de borradores de correo electrónico.

Esta arquitectura responde a requisitos especialmente relevantes en entornos institucionales, como el control del flujo, la privacidad de la información, la trazabilidad de las respuestas y la supervisión humana de las acciones propuestas. De este modo, el asistente no se concibe únicamente como una interfaz conversacional para responder preguntas, sino como un sistema capaz de transformar una consulta documental en una respuesta fundamentada y, cuando procede, en una acción operativa supervisada. La aportación del trabajo reside, por tanto, en articular dentro de una misma solución conocimiento documental, mejora de la recuperación, decisión sobre el flujo de ejecución y apoyo accionable al usuario.

Para evaluar el rendimiento de la propuesta, se definirá un conjunto de datos de prueba compuesto por documentación institucional y un conjunto de consultas representativas del dominio. Estas consultas cubrirán distintos escenarios de uso, incluyendo preguntas informativas, consultas sobre procedimientos y peticiones orientadas a la generación de una acción posterior, como la redacción de un correo formal. Sobre este conjunto se comparará el comportamiento de un *baseline* basado en búsqueda semántica con una variante mejorada basada en búsque-

da híbrida y *reranking*, considerando tanto métricas de recuperación como la fundamentación de las respuestas generadas y la utilidad de las salidas producidas.

El asistente no se limita a responder preguntas, sino que puede facilitar tareas útiles para el usuario, como la generación o el envío supervisado de correos electrónicos. Por último, el trabajo incluye una vertiente experimental centrada en la mejora del componente de recuperación de información, mediante la comparación entre un *baseline* inicial y una estrategia mejorada basada en recuperación híbrida y *reranking*.

La Figura 1 muestra la arquitectura general del asistente propuesto. El sistema se organiza en dos grandes bloques principales. Por una parte, se distingue una fase offline de preparación del corpus, en la que las fuentes documentales se recopilan, transforman, segmentan e indexan para su posterior recuperación. Por otra parte, se define una fase online de ejecución, en la que el usuario interactúa con el asistente a través de una interfaz conversacional.

En esta fase online, el agente orquestador actúa como componente central de coordinación. Su función es interpretar la consulta del usuario, decidir qué capacidades deben activarse y coordinar el uso de las herramientas disponibles. Entre estas herramientas se incluye el componente de recuperación documental, encargado de consultar la base documental formada por Qdrant y BM25, así como una herramienta accionable orientada a la generación de borradores de correo. Finalmente, el modelo de lenguaje utiliza la información recuperada y el flujo decidido por el orquestador para generar la respuesta final o el borrador correspondiente.

Además, la arquitectura incorpora una capa de observabilidad basada en Phoenix y Open-Telemetry. Esta capa permite registrar trazas del comportamiento del sistema, revisar los documentos recuperados, analizar las llamadas al modelo y facilitar la depuración del pipeline durante las fases de desarrollo y evaluación.

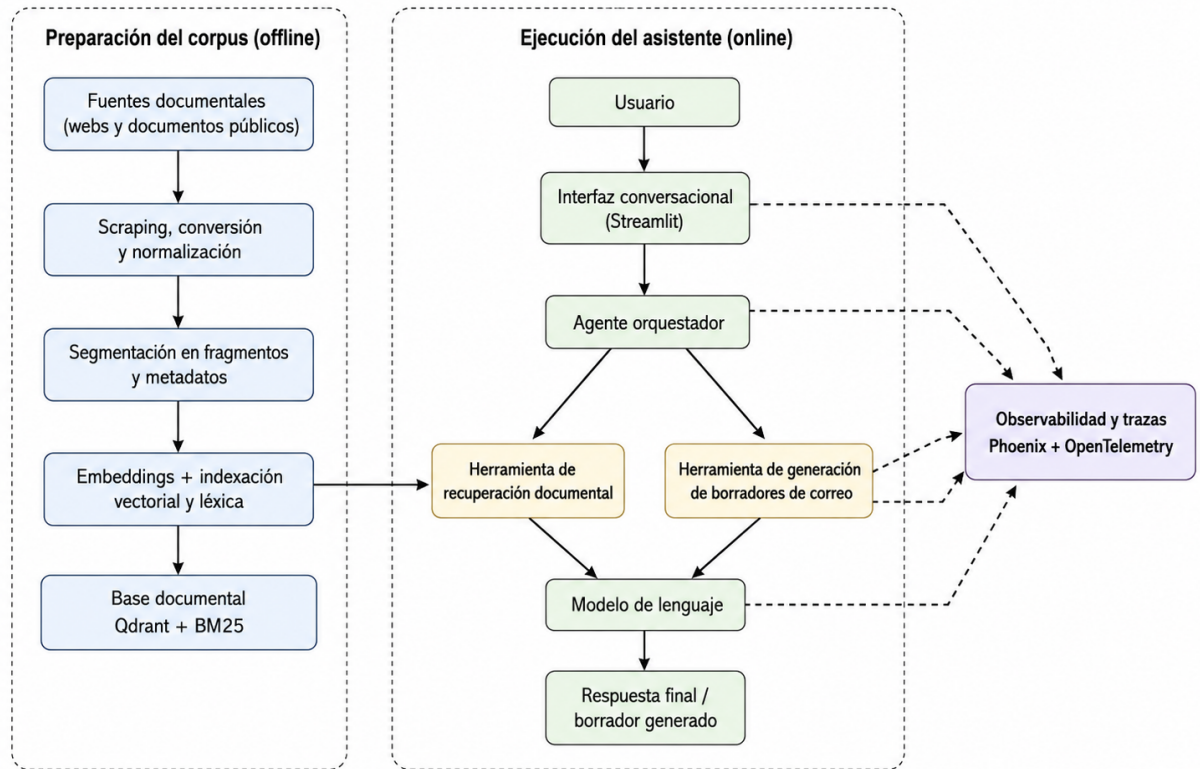


Figura 1: Arquitectura general del asistente institucional propuesto.

1.3. Estructura del trabajo

La memoria se organiza en varios capítulos que permiten abordar de forma progresiva tanto la definición del problema y la planificación del trabajo como su fundamentación teórica, su desarrollo técnico y su validación experimental.

De forma más concreta, la estructura del documento es la siguiente:

- **Capítulo 1. Introducción.** Presenta el contexto y la motivación del trabajo, el planteamiento general de la solución y la organización de la memoria.
- **Capítulo 2. Contexto y estado del arte.** Revisa los antecedentes más relevantes sobre agentes basados en modelos de lenguaje, orquestación, uso de herramientas, sistemas de recuperación aumentada, mejora del *retrieval* y asistentes institucionales especializados.
- **Capítulo 3. Objetivos y metodología.** Define los objetivos generales y específicos del trabajo, los criterios de comprobación planteados, la metodología seguida, la planificación, los riesgos identificados y los recursos necesarios.

- **Capítulo 4. Diseño e implementación de la propuesta.** Describe la arquitectura general del asistente institucional, el papel del agente orquestador, el sistema de recuperación documental, la integración de herramientas accionables y las principales decisiones técnicas adoptadas durante el desarrollo del prototipo.
- **Capítulo 5. Evaluación experimental y discusión de resultados.** Presenta el conjunto de evaluación, el *baseline* de recuperación, la variante mejorada basada en búsqueda híbrida y *reranking*, las métricas empleadas, los resultados obtenidos y su análisis en relación con los objetivos planteados y las limitaciones del sistema.
- **Capítulo 6. Conclusiones y trabajo futuro.** Resume las principales aportaciones del trabajo, valora el grado de cumplimiento de los objetivos planteados y expone posibles líneas de mejora o ampliación futura.

Capítulo 2

Contexto y estado del arte

En este capítulo se describe el contexto del trabajo y se revisa el estado del arte de las tecnologías y enfoques más relevantes para la propuesta. Además, se analiza el encaje de la solución planteada dentro de la literatura revisada y se exponen las principales conclusiones obtenidas.

2.1. Contexto

En los últimos años, el desarrollo de la inteligencia artificial generativa ha impulsado la aparición de agentes conversacionales cada vez más capaces de desenvolverse en escenarios reales. Una de las líneas más relevantes en esta evolución ha sido la incorporación de mecanismos de acceso a conocimiento externo, de modo que los modelos pueden consultar documentación, bases de datos y servicios externos para fundamentar mejor sus respuestas. En este contexto, los enfoques basados en *Retrieval-Augmented Generation* (RAG) (Lewis y col., 2020) han adquirido un papel central, al permitir combinar las capacidades generativas de los modelos con la recuperación de documentación relevante.

Al mismo tiempo, los modelos de lenguaje han dejado de concebirse únicamente como interfaces conversacionales y han pasado a integrarse en sistemas capaces de apoyar flujos de trabajo más amplios. Esta evolución resulta especialmente significativa en entornos institucionales, donde las consultas de los usuarios no siempre finalizan en una simple respuesta informativa, sino que suelen desembocar en una acción posterior, como la redacción de un correo o la realización de un trámite. En este tipo de contextos aparecen además exigencias adicionales, ya que el sistema debe operar sobre documentación interna, respetar restricciones de acceso a la información y ofrecer un mayor grado de control sobre su comportamiento.

2.2. Estado del arte

La literatura reciente ha explorado con especial interés las arquitecturas de agentes y los mecanismos de orquestación. En esta línea, ReAct (Yao y col., 2023) plantea una combinación entre razonamiento y acción, mientras que trabajos como AgentVerse (Chen y col., 2024b), MetaGPT (Hong y col., 2024) y AutoGen (Wu y col., 2024) avanzan hacia esquemas de colaboración y coordinación entre agentes. Más recientemente, Dang et al. (Dang y col., 2025) abordan la orquestación multiagente como un mecanismo dinámico de coordinación. De forma complementaria, otra línea de trabajo relevante se centra en el uso de herramientas y APIs externas por parte de modelos de lenguaje. Toolformer (Schick y col., 2023) introduce la capacidad de aprender cuándo y cómo invocar herramientas, mientras que Gorilla (Patil y col., 2024) y ToolLLM (Qin y col., 2024) exploran la conexión de modelos de lenguaje con APIs reales. En esta misma dirección, API-Bank (Li y col., 2023) propone un benchmark específico para evaluar sistemas aumentados con herramientas. Asimismo, los sistemas de Retrieval-Augmented Generation fueron introducidos por Lewis et al. (Lewis y col., 2020) como un enfoque para combinar modelos generativos con recuperación de conocimiento externo. Trabajos posteriores como REPLUG (Shi y col., 2024), Self-RAG (Asai y col., 2024) y Adaptive-RAG (Jeong y col., 2024) han profundizado en distintas formas de integrar, controlar o adaptar el proceso de recuperación. Finalmente, también se han propuesto asistentes especializados para dominios institucionales, universitarios o cerrados. Marcel (Trientes y col., 2025) se orienta al soporte conversacional de estudiantes universitarios, URASys (Nguyen y col., 2025) aborda consultas ambiguas o no respondibles en contextos académicos, y HyPA-RAG (Kalra y col., 2025) aplica recuperación híbrida y adaptación de parámetros en un dominio especializado.

Estas líneas resultan especialmente relevantes porque la propuesta no se limita a diseñar un sistema conversacional genérico, sino un asistente institucional con capacidad para consultar documentación interna, utilizar herramientas accionables y transformar una respuesta fundamentada en una actuación útil para el usuario. Además, el trabajo incorpora una contribución experimental específica centrada en la mejora del componente de recuperación de información mediante una estrategia de recuperación híbrida y *reranking*. Por ello, este estado del arte no solo revisa los fundamentos técnicos del sistema, sino también sitúa la propuesta en relación con soluciones institucionales existentes y con las principales líneas de mejora del retrieval.

2.2.1. Agentes y orquestación

Una de las líneas más influyentes en este ámbito es la que entiende al agente basado en modelos de lenguaje no como un simple generador de texto, sino como una entidad capaz de razonar, planificar y actuar sobre el entorno. En esta dirección, Yao et al. (Yao y col., 2023) proponen ReAct, un enfoque que combina trazas de razonamiento con acciones explícitas ejecutadas sobre fuentes externas o entornos interactivos. Para el contexto de este trabajo, ReAct resulta especialmente relevante porque introduce una base conceptual clara para el diseño de un agente orquestador que no solo responde, sino que decide qué hacer, qué consultar y en qué orden ejecutar cada paso.

Sobre esta idea inicial, la investigación posterior ha ampliado el foco desde agentes individuales hacia sistemas compuestos por múltiples agentes especializados, como AgentVerse (Chen y col., 2024b), un marco de colaboración multiagente en el que distintos agentes cooperan para resolver tareas. Los resultados descritos por los autores muestran que, en determinados escenarios, un conjunto coordinado de agentes puede superar a un único agente generalista.

Hong et al. (Hong y col., 2024) proponen MetaGPT que frente a aproximaciones más informales, introduce una organización más estructurada del trabajo entre agentes, de forma que cada componente participa en una fase concreta del proceso y verifica resultados intermedios. Sugiere que la división explícita de responsabilidades puede reducir inconsistencias y facilitar la incorporación de nuevas capacidades sin rediseñar completamente el sistema.

Por su parte, Wu et al. (Wu y col., 2024) presentan AutoGen, uno de los elementos más destacables es que los agentes pueden operar en distintos modos, combinando modelos de lenguaje, intervención humana y uso de herramientas externas. Esta visión encaja con el enfoque de este proyecto, en el que se persigue una arquitectura en la que un agente orquestador coordine módulos y capacidades desacopladas dentro de un entorno controlado.

Más recientemente, Dang et al. (Dang y col., 2025) plantean un esquema centralizado en el que un orquestador dirige de forma adaptativa a los distintos agentes según el estado de la tarea. Aportan evidencias de que la orquestación debe entenderse como un mecanismo de coordinación capaz de reorganizar prioridades, seleccionar agentes adecuados y ajustar el flujo de ejecución en función del contexto.

2.2.2. Uso de herramientas e integración de capacidades

Una de las limitaciones más relevantes de los modelos de lenguaje es su dificultad para interactuar de forma fiable con recursos externos. Por este motivo, una línea de investigación

cada vez más importante se centra en el *tool use*, entendido como la capacidad de un modelo para decidir cuándo utilizar una herramienta y cómo incorporar el resultado obtenido a la respuesta final.

Uno de los trabajos de referencia en esta línea es Toolformer (Schick y col., 2023). Su contribución principal es demostrar que el modelo no solo puede ejecutar llamadas a APIs, sino también aprender a decidir en qué momento conviene hacerlo y qué argumentos debe pasar. Esta propuesta aporta una base conceptual sólida para entender la invocación de herramientas como una capacidad integrada dentro del comportamiento del agente.

Una contribución complementaria es ToolLLM, presentada por Qin et al. (Qin y col., 2024). En este caso, los autores proponen entrenar y evaluar modelos de lenguaje en escenarios de uso de herramientas sobre un conjunto masivo de APIs reales. Para el contexto de este trabajo, ToolLLM refuerza la idea de que una arquitectura modular debe diseñarse pensando en la extensibilidad, evitando acoplamientos fuertes entre el agente y cada herramienta concreta.

Junto a estas propuestas, también resulta necesario considerar trabajos centrados en su evaluación. En este sentido, Li et al. (Li y col., 2023) presentan API-Bank, un *benchmark* en el que se evalúan aspectos como la planificación, la recuperación de la API correcta y la ejecución de llamadas.

2.2.3. RAG

La técnica de *Retrieval-Augmented Generation* (RAG) surge como una respuesta a una de las limitaciones fundamentales de los modelos de lenguaje: su dependencia exclusiva del conocimiento almacenado en los parámetros del modelo. Aunque este conocimiento permite obtener un buen rendimiento en múltiples tareas, presenta problemas importantes cuando se requiere acceder a información específica o perteneciente a un dominio concreto.

En este contexto, Lewis et al. (Lewis y col., 2020) introducen RAG como un marco que combina un modelo generativo con una memoria externa, accesible mediante un mecanismo de recuperación. Su relevancia radica en que permite paliar las limitaciones de los modelos de lenguaje en dominios específicos al proporcionarles evidencia externa actualizada. No obstante, el uso de estos sistemas plantea retos significativos en el componente de recuperación: la búsqueda semántica simple a menudo presenta dificultades para capturar términos técnicos precisos, gestionar la ambigüedad de las consultas o garantizar que los documentos recuperados sean los más relevantes para la generación final. Afrontar estos retos justifica la necesidad de evolucionar el modelo hacia un componente de recuperación de información más robusto, que actúe como base sólida para la consulta y fundamentación de las respuestas.

La investigación posterior ha explorado distintas variantes orientadas a hacer el uso de RAG más flexible y aplicable en sistemas reales. En esta línea, Shi et al. (Shi y col., 2024) proponen REPLUG, donde tratan al modelo de lenguaje como una caja negra y utilizan un recuperador ajustable cuyos documentos se añaden directamente al *prompt* del modelo. Demostraron que los beneficios de RAG no dependen necesariamente de modificar la arquitectura del modelo, sino que pueden alcanzarse mediante mecanismos externos de recuperación e integración de contexto.

Asai et al. (Asai y col., 2024) presentan Self-RAG, un enfoque en el que el modelo aprende no solo a recuperar información, sino también a reflexionar sobre la necesidad de dicha recuperación y a criticar la calidad de la respuesta generada. Esta contribución desplaza de la idea de un RAG estático hacia uno más adaptativo y autorreflexivo.

Jeong et al. (Jeong y col., 2024) proponen Adaptive-RAG, una arquitectura que selecciona dinámicamente la estrategia de recuperación más adecuada en función de la complejidad de la pregunta. De este modo, la literatura reciente sugiere que la calidad de un sistema RAG depende de la capacidad de decidir cómo y cuándo recuperar evidencia.

Una línea de investigación especialmente importante es la relacionada con la privacidad en sistemas RAG. En este sentido, Zeng et al. (Zeng y col., 2024) estudian cómo un sistema RAG puede exponer contenido de la base de recuperación y evidencian que el uso de información privada no solo aporta beneficios en términos de utilidad, sino que también exige nuevas medidas de protección. Refuerza el considerar la privacidad no como una característica accesorio, sino como una restricción de diseño de primer nivel.

2.2.4. Evaluación y mejora concreta del retrieval

Esta cuestión resulta especialmente relevante, ya que una de sus contribuciones previstas consiste precisamente en proponer y evaluar una mejora concreta sobre el componente de recuperación frente a un *baseline* inicial.

Un punto de partida fundamental en esta línea es el trabajo de Karpukhin et al. (Karpukhin y col., 2020) sobre *Dense Passage Retrieval* (DPR). DPR plantea aprender representaciones vectoriales en las que cada pregunta quede cerca de los documentos más relevantes, de manera que sea posible recuperarlos de forma eficiente incluso en grandes colecciones de documentos.

A partir de esta base, varios trabajos han abordado la optimización del entrenamiento de recuperadores densos. Entre ellos, Qu et al. (Qu y col., 2021) proponen RocketQA, que introduce mejoras específicas como la selección de negativos poco informativos o la existencia de

falsos negativos en el entrenamiento. Esta aportación demuestra que una mejora del *retrieval* no tiene por qué implicar rediseñar todo el pipeline, sino que puede consistir en una forma específica el entrenamiento o la selección de evidencia recuperada.

Nogueira et al. (Nogueira y col., 2020b) demuestran que un modelo generativo puede emplearse eficazmente para calcular relevancia documental y reordenar documentos previamente recuperados. Esta estrategia resulta atractiva porque permite aumentar la calidad del contexto recuperado sin comprometer excesivamente la arquitectura.

No obstante, mejorar el *retrieval* no es suficiente si no se dispone de criterios adecuados para medir su impacto. En este punto cobra especial relevancia el trabajo de Saad-Falcon et al. (Saad-Falcon y col., 2024) sobre ARES, el cual propone evaluar separadamente distintas dimensiones del sistema, como la relevancia del contexto recuperado y la relevancia global de la respuesta generada. En lugar de limitar la evaluación a métricas, refuerza la idea de que conviene analizar también la calidad del contexto recuperado.

2.2.5. Asistentes institucionales

Debido al contexto de este trabajo, interesa analizar propuestas que hayan abordado asistentes en contextos institucionales y trabajos que hayan explorado el uso de herramientas en lenguaje natural.

En el ámbito universitario, Trienes et al. (Trienes y col., 2025) presentan Marcel, un agente conversacional ligero y de código abierto orientado al soporte de estudiantes, su alcance permanece centrado en la respuesta a consultas de admisión. La solución combina un módulo de *Frequently Asked Questions* con un esquema de *Retrieval-Augmented Generation* para fundamentar las respuestas con información verificable.

Una propuesta más robusta en escenarios cerrados es URASys, de Nguyen et al. (Nguyen y col., 2025), que plantea un sistema unificado para responder preguntas estándar y ambiguas en tareas de *academic advising*. Es especialmente relevante porque muestra que, en contextos institucionales, no basta con recuperar información, sino que también es necesario manejar ambigüedad y reducir alucinaciones. No obstante, no aborda de forma central la integración de herramientas.

Más allá del entorno universitario, encontramos varias propuestas. Liu et al. (Liu y col., 2025) proponen WorkTeam, un marco multiagente para transformar instrucciones en lenguaje natural en flujos de trabajo estructurados en entornos empresariales. Su arquitectura se compone de tres agentes especializados: un supervisor que planifica, un orquestador que organiza componentes, y un agente *filler* que completa parámetros a partir de documentación relevante. Este

trabajo transforma una instrucción textual a una acción compuesta sobre herramientas.

En una línea cercana al uso de herramientas, Zhang et al. (Zhang y col., 2025) presentan AskToAct. La idea central del trabajo consiste en utilizar los propios parámetros de las herramientas como representación explícita de la intención del usuario, generando automáticamente datos de entrenamiento a partir de consultas incompletas. Esta aportación refleja que, para ejecutar correctamente una herramienta, el sistema debe ser capaz de detectar qué información falta y solicitarla de manera adecuada.

Por último, Kalra et al. (Kalra y col., 2025) presentan HyPA-RAG, un sistema de *Retrieval-Augmented Generation* adaptativo para aplicaciones legales. La solución incorpora un clasificador de complejidad de la consulta para ajustar parámetros dinámicamente, una recuperación híbrida que combina métodos basados en grafos de conocimiento, y un marco de evaluación adaptado al dominio. Este trabajo es especialmente relevante para la parte investigadora, ya que demuestra que una mejora explícita del módulo de recuperación puede traducirse en ganancias en exactitud de recuperación, fidelidad y precisión contextual.

2.3. Encaje de la propuesta

La propuesta de este TFM se sitúa en la intersección entre asistentes institucionales especializados, sistemas de uso de herramientas y mejora experimental del componente de recuperación de información. Por una parte, se plantea un asistente institucional especializado orientado a resolver consultas sobre documentación interna. Por otra, incorpora una funcionalidad accionable: la generación y eventual envío supervisado de correos electrónicos como paso operativo posterior a la consulta. Finalmente incorpora una contribución experimental específica sobre el componente de recuperación de información basada en recuperación híbrida y *reranking*.

La propuesta busca cerrar el ciclo entre consulta y acción, transformando una necesidad informativa en una respuesta fundamentada y, cuando procede, en una actuación útil, trazable y supervisada. Esta es precisamente la aportación diferencial del trabajo: combinar en una misma arquitectura capacidades que en la literatura suelen abordarse por separado, como la recuperación documental, la mejora del ranking, la orquestación del flujo y el uso controlado de herramientas.

Comparada con Marcel, nuestra propuesta comparte el interés por un asistente universitario basado en RAG, pero se diferencia en dos aspectos fundamentales. En primer lugar, el caso de uso no se restringe a admisión ni a orientación externa, sino que se orienta a soporte institucional sobre documentación y procedimientos internos. En segundo lugar, el sistema no

se limita a responder preguntas, sino que incorpora una transición explícita desde la evidencia documental hacia una acción operativa, en este caso la redacción o envío supervisado de correos formales.

En relación con URASys, la propuesta coincide en la importancia de operar en entornos cerrados, reducir alucinaciones y tratar consultas difíciles. No obstante, mientras URASys concentra su aportación en la robustez del *question answering* frente a ambigüedad y no respondibles, este proyecto amplía el alcance hacia una arquitectura con capacidad de acción.

WorkTeam transforma instrucciones en *workflows* generales de negocio mediante una arquitectura multiagente de orquestación y relleno de parámetros. Esta propuesta, en cambio, parte de un escenario institucional centrado en la consulta documental y utiliza la acción como extensión natural de una respuesta fundamentada por RAG.

Finalmente, HyPA-RAG constituye el antecedente más cercano en la parte investigadora, ya que también propone mejorar el componente de recuperación de información mediante recuperación híbrida y ajuste adaptativo. No obstante, la diferencia principal es el contexto ya que HyPA-RAG se presenta como una solución especializada para consultas legales y de política pública.

Por tanto, la aportación diferencial radica en la combinación de elementos que, en los trabajos revisados, aparecen habitualmente por separado.

2.4. Conclusiones

La revisión realizada permite extraer varias conclusiones relevantes. En primer lugar, las aportaciones respaldan el diseño de una arquitectura modular con un agente orquestador que coordine herramientas y capacidades especializadas, especialmente cuando se busca extensibilidad, trazabilidad y control en entornos privados.

En segundo lugar, integrar herramientas de forma efectiva exige resolver cuestiones fundamentales: decidir cuándo es necesario recurrir a una capacidad externa, seleccionar la herramienta adecuada, construir correctamente la llamada y utilizar el resultado obtenido de manera coherente dentro del proceso de generación.

En tercer lugar, los trabajos revisados sobre *Retrieval-Augmented Generation* muestran que no siempre basta con recuperar información relevante, sino que también es necesario hacerlo mediante una arquitectura controlada y alineada con los requisitos de privacidad.

Asimismo, la literatura permite identificar varias estrategias principales para mejorar el componente de recuperación. Destacan los enfoques en varias etapas, donde una recuperación inicial eficiente se complementa con un *reranker* más preciso. La investigación reciente sub-

raya que la evaluación del *retrieval* debe abordarse de forma explícita, considerando no solo cuántos documentos relevantes se recuperan, sino también cómo afecta dicha recuperación a la fidelidad y utilidad de la respuesta final. Por ello, este bloque del estado del arte proporciona una base directa para la parte experimental, justificando la comparación entre un *baseline* de recuperación semántica y una mejora concreta basada en recuperación híbrida y *reranking*.

En conjunto, los trabajos revisados muestran la existencia de asistentes especializados en dominios institucionales o cerrados, el interés creciente por la coordinación de herramientas, y la mejora del propio componente de recuperación de información como vía activa de investigación aplicada. Sin embargo, estas líneas suelen aparecer separadas y este vacío justifica el interés de una propuesta que combine consulta documental fundamentada, uso de herramientas orientadas a la acción y mejora experimental del componente de recuperación de información dentro de un mismo asistente institucional.

Capítulo 3

Objetivos y Metodología

En este capítulo, se presentarán los objetivos generales y específicos del trabajo, junto con los criterios definidos para comprobar su cumplimiento. Posteriormente, se describirá la metodología adoptada para el desarrollo del proyecto, así como la planificación prevista, los riesgos identificados y los recursos necesarios para su realización.

3.1. Objetivos Generales

El objetivo general es desarrollar y evaluar un asistente institucional, capaz de consultar documentación interna mediante un sistema *Retrieval-Augmented Generation* (RAG) y de apoyar acciones operativas mediante herramientas integradas, con el fin de mejorar su utilidad en un entorno controlado.

De forma más específica, el trabajo busca validar una arquitectura en la que estas capacidades no se traten como módulos aislados, sino como partes coordinadas de un mismo flujo: recuperación de evidencia, mejora del ranking, generación de respuesta, decisión orquestada y acción supervisada.

A mayores se persigue analizar en qué medida una mejora del componente de recuperación, basada en recuperación híbrida y *reranking*, produce un efecto observable sobre la calidad de la evidencia recuperada y de las respuestas generadas, en comparación con un *baseline* de recuperación semántica simple.

3.2. Objetivos Específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

1. **Identificar** los requisitos funcionales y no funcionales de un asistente institucional privado, atendiendo a aspectos como consulta documental, fundamentación de respuestas, trazabilidad, control del flujo y uso supervisado de herramientas.
2. **Diseñar** una arquitectura modular basada en un agente orquestador que coordine la recuperación de información, la generación de respuestas y la invocación de herramientas externas.
3. **Desarrollar** un prototipo funcional capaz de procesar consultas en lenguaje natural sobre documentación interna y generar respuestas apoyadas en evidencia recuperada.
4. **Integrar** una funcionalidad accionable basada en herramientas, centrada en la generación de correos electrónicos formales a partir de la información obtenida durante la consulta.
5. **Establecer** un *baseline* de recuperación basado en búsqueda semántica simple para disponer de una referencia inicial de funcionamiento.
6. **Implementar** una variante mejorada del componente de recuperación de información mediante una estrategia de recuperación híbrida y *reranking*.
7. **Definir** un conjunto de casos de prueba representativos del dominio institucional, incluyendo consultas informativas, consultas sobre procedimientos y consultas orientadas a acción.
8. **Comparar** el rendimiento del *baseline* y de la variante mejorada mediante métricas de recuperación, fundamentación de respuestas, utilidad de la salida generada y coste temporal del sistema.
9. **Evaluar** a través de métricas en qué medida la mejora introducida en el componente de recuperación repercute en la calidad global del asistente y en su capacidad para ofrecer respuestas útiles y operativamente aprovechables.
10. **Analizar** las fortalezas, limitaciones y posibles líneas de mejora de la solución propuesta en el contexto de asistentes institucionales privados y uso de herramientas.

3.3. Criterios de comprobación de los objetivos

Con el fin de que los objetivos anteriores puedan verificarse de forma objetiva, se emplearán los siguientes criterios de comprobación:

- El conjunto de evaluación deberá contener al menos 25 casos de prueba representativos del dominio institucional, incluyendo preguntas respondibles y preguntas no respondibles a partir del corpus disponible.
- El corpus documental deberá estar formado por documentos públicos del dominio institucional, procesados y segmentados en fragmentos adecuados para su recuperación posterior.
- La comparación entre el *baseline* y la variante mejorada se realizará mediante métricas de recuperación como Hit@1, Hit@3 y MRR.
- Se considerará que la mejora del componente de recuperación produce un efecto observable si la variante mejorada obtiene, al menos, una mejora de 3 puntos porcentuales en Hit@1 o Hit@3, o una mejora relativa del 5 % en MRR respecto al *baseline*. En este contexto, Hit@1 indica si la fuente esperada aparece en la primera posición, Hit@3 indica si aparece entre los tres primeros resultados y MRR mide la posición media recíproca de la primera fuente relevante recuperada.
- La calidad de las respuestas generadas se analizará mediante la cobertura de hechos esperados en las preguntas respondibles, comparando el comportamiento del flujo determinista y del orquestador.
- En las preguntas no respondibles, se evaluará la capacidad del sistema para abstenerse de responder cuando no exista evidencia suficiente en el corpus documental.
- En los casos orientados a acción, se evaluará la capacidad del orquestador para detectar la intención de generación de correo y producir un borrador adecuado, sin realizar el envío real del mensaje.
- La evaluación funcional del orquestador comprobará si el sistema selecciona el flujo adecuado en consultas simples, consultas multi-documento, preguntas comparativas, preguntas no respondibles y solicitudes de generación de borradores.

3.4. Metodología del trabajo

En el desarrollo de este se adoptará una metodología ágil basada en Scrum, al considerarse un enfoque adecuado para organizar un proyecto tecnológico con una parte aplicada y otra parte evaluativa. Esta elección se justifica por su flexibilidad, su carácter iterativo y su capacidad

para adaptarse a cambios en los requisitos, en las decisiones de diseño o en los resultados obtenidos durante la implementación y la evaluación del sistema.

En este caso, el trabajo no consiste únicamente en desarrollar un prototipo funcional, sino también en analizar comparativamente el comportamiento de distintas configuraciones del componente de recuperación de información. Por ello, Scrum se empleará como marco de organización y seguimiento del desarrollo, permitiendo dividir el trabajo en incrementos manejables, revisar periódicamente el progreso y ajustar decisiones conforme avance el proyecto.

3.4.1. Scrum

Scrum es un marco de trabajo ágil orientado al desarrollo incremental de productos. Su funcionamiento se basa en iteraciones cortas y planificadas, denominadas *sprints*, al final de las cuales se obtiene un incremento funcional del sistema. Este enfoque resulta apropiado, ya que permite avanzar de forma progresiva desde la definición del problema hasta la implementación del prototipo y su evaluación experimental.

3.4.2. Roles

Dado el contexto académico del trabajo, los roles de Scrum se adaptarán de forma simplificada:

- **Product Owner:** asumido por la dirección, al encargarse de supervisar el enfoque general del trabajo, validar su orientación y priorizar los elementos más relevantes del proyecto.
- **Scrum Master:** también asumido de forma parcial por la dirección, en la medida en que orienta el desarrollo, ayuda a resolver bloqueos y supervisa la coherencia metodológica del trabajo.
- **Equipo de desarrollo:** asumido por la estudiante, responsable del análisis, diseño, implementación, experimentación y redacción de la memoria.

3.4.3. Artefactos

Los principales artefactos de trabajo serán los siguientes:

- **Product Backlog:** listado priorizado de tareas y objetivos del proyecto, incluyendo decisiones de arquitectura, preparación documental, implementación de componentes, diseño de experimentos y redacción.

- **Sprint Backlog:** subconjunto de tareas seleccionadas para cada sprint.
- **Incremento:** resultado funcional o evaluable obtenido al final de cada sprint, como puede ser una primera arquitectura definida, un *baseline* operativo o una versión mejorada del sistema.

3.4.4. Eventos

El trabajo se organizará mediante los eventos habituales de Scrum, adaptados a un proyecto individual:

- **Sprint Planning:** definición de las tareas a abordar en cada iteración.
- **Seguimiento:** revisión periódica del progreso, dificultades encontradas y ajustes necesarios.
- **Sprint Review:** revisión del resultado obtenido al final de cada sprint.
- **Sprint Retrospective:** valoración de los aspectos que han funcionado correctamente y de los elementos a mejorar en la siguiente iteración.

3.4.5. Adaptación al proyecto

Aunque Scrum se concibe habitualmente para equipos de desarrollo, su estructura puede adaptarse a un trabajo individual. Los *sprints* no se orientarán únicamente a implementar funcionalidades, sino también a producir resultados parciales verificables.

De este modo, la metodología de desarrollo permitirá combinar dos dimensiones del trabajo: por una parte, la construcción iterativa del asistente institucional privado; por otra, la comparación entre un *baseline* de recuperación semántica y una variante mejorada basada en recuperación híbrida y *reranking*.

3.5. Riesgos y planes de contingencia

Durante el desarrollo resulta necesario identificar posibles riesgos técnicos, metodológicos y de planificación que puedan afectar al alcance, a la calidad de la solución o al calendario previsto. Dado que el proyecto combina implementación de un prototipo funcional con una evaluación experimental comparativa, la gestión de riesgos adquiere especial relevancia para asegurar la viabilidad del trabajo.

A continuación, se describen los principales riesgos identificados y las medidas de contingencia previstas para reducir su impacto:

1. **Dificultades en la integración de componentes heterogéneos.** La integración entre el modelo de lenguaje, el componente de recuperación, la base documental y la funcionalidad accionable puede presentar incompatibilidades técnicas o requerir más esfuerzo de implementación del previsto. Como plan de contingencia, se priorizará una arquitectura modular y desacoplada, permitiendo validar cada componente por separado antes de su integración completa.
2. **Calidad insuficiente de la base documental o del conjunto de evaluación.** El rendimiento del asistente depende en gran medida de la calidad del corpus documental y de la representatividad de los casos de prueba. Un conjunto poco adecuado podría dificultar tanto el funcionamiento del sistema como la evaluación de la mejora propuesta. Como medida de contingencia, se realizará una selección y revisión temprana del corpus y de las consultas de prueba, garantizando que cubran los escenarios principales del dominio.
3. **Resultados experimentales poco concluyentes.** Existe la posibilidad de que la mejora basada en recuperación híbrida y *reranking* no produzca diferencias suficientemente claras respecto al *baseline*, o que dichas diferencias no sean tan amplias como se esperaba inicialmente. Como plan de contingencia, el trabajo no se orientará a demostrar una mejora absoluta, sino a realizar una comparación controlada y rigurosa entre ambas configuraciones.
4. **Incremento excesivo de la latencia del sistema.** La incorporación de una segunda etapa de recuperación o de un mecanismo de *reranking* puede aumentar el tiempo de respuesta del asistente y comprometer su utilidad práctica como prototipo. Para mitigar este riesgo, se medirá la latencia desde las primeras versiones funcionales y se limitará el número de documentos candidatos o el coste de los modelos auxiliares en caso necesario.
5. **Cambios en el alcance o en las decisiones técnicas durante el desarrollo.** Como sucede en muchos proyectos de desarrollo e investigación aplicada, algunas decisiones inicialmente previstas pueden necesitar ajustes conforme avance la implementación o la evaluación. Como medida de contingencia, se adoptará una planificación iterativa basada en Scrum, de forma que cada sprint permita revisar el estado del trabajo, reordenar prioridades y acotar el alcance sin perder coherencia con los objetivos definidos.

6. **Retrasos en el calendario previsto.** La simultaneidad entre implementación, experimentación y redacción de la memoria puede provocar desviaciones temporales respecto a la planificación inicial. Para reducir este riesgo, se organizará el trabajo en incrementos parciales y se reservará una fase final específica para revisión, análisis de resultados y redacción.
7. **Limitaciones derivadas del carácter individual del proyecto.** Al tratarse de un TFM desarrollado de forma individual, tanto el análisis como la implementación, la evaluación y la documentación recaen sobre una única persona, lo que puede ralentizar el avance en fases de mayor carga técnica. Como contingencia, se mantendrá una planificación realista, con objetivos acotados y verificables, evitando incorporar funcionalidades adicionales que no resulten imprescindibles para demostrar la contribución principal del trabajo.

3.6. Planificación del trabajo

De manera general, cada sprint tendrá una duración aproximada de entre dos y tres semanas, como se muestra en la Figura 2, aunque esta distribución podrá adaptarse en función de la complejidad real de las tareas y de las incidencias surgidas durante el desarrollo. Al final de cada iteración se realizará una revisión del avance conseguido y una replanificación de las tareas pendientes, con el fin de mantener el proyecto alineado con los objetivos establecidos.

3.6.1. Sprint 1: análisis y definición del problema

En esta primera fase se concretará el alcance del trabajo, se revisará el estado del arte ya recopilado y se terminarán de definir los requisitos funcionales y no funcionales del asistente institucional privado. Asimismo, se delimitará el caso de uso principal, se identificarán las capacidades mínimas del prototipo y se establecerán los criterios de evaluación que se utilizarán en la parte experimental.

Durante esta fase también se seleccionarán las herramientas y tecnologías necesarias para implementar el sistema, así como el enfoque general de la arquitectura.

3.6.2. Sprint 2: diseño de la arquitectura y preparación del entorno de trabajo

El segundo sprint estará centrado en el diseño técnico del sistema. En esta etapa se definirá la arquitectura modular del asistente, identificando los componentes principales, sus responsabilidades y el flujo de interacción entre ellos. En particular, se detallará el papel del agente

orquestador, el componente de recuperación documental, el modelo generativo y la funcionalidad de herramienta integrada.

Además, se preparará el entorno de desarrollo, se seleccionará o estructurará el corpus documental inicial y se organizará el material necesario para la construcción posterior del prototipo.

3.6.3. Sprint 3: implementación del *baseline* del asistente

En este sprint se desarrollará una primera versión funcional del sistema, que actuará como *baseline*. Esta configuración inicial permitirá procesar consultas en lenguaje natural, recuperar documentos relevantes mediante búsqueda semántica simple y generar respuestas apoyadas en la evidencia recuperada.

El objetivo de esta fase será disponer de una referencia operativa sobre la que poder comparar posteriormente la mejora investigadora del trabajo. Al finalizar este sprint deberá existir una primera versión ejecutable del asistente, aunque todavía limitada a su funcionalidad esencial.

3.6.4. Sprint 4: integración de la funcionalidad accionable

Una vez disponible el *baseline*, el siguiente sprint se centrará en ampliar el sistema con una capacidad accionable. En concreto, se integrará la generación de borradores de correos electrónicos formales a partir de la consulta del usuario y de la información recuperada desde la base documental.

Esta fase permitirá que el asistente no solo responda preguntas, sino que también apoye una actuación posterior útil dentro del entorno institucional. Al finalizar este sprint, el prototipo deberá ser capaz de producir una salida operativa básica en los casos orientados a acción.

3.6.5. Sprint 5: implementación de la mejora del componente de recuperación de información

El quinto sprint estará orientado a la principal aportación experimental del TFM. En esta etapa se implementará una variante mejorada del componente de recuperación de información basada en recuperación híbrida y *reranking*, manteniendo la posibilidad de compararla con el *baseline* definido anteriormente. El objetivo será introducir una mejora concreta y controlada sobre el proceso de recuperación documental, sin modificar de forma radical el resto del sistema.

3.6.6. Sprint 6: diseño de la evaluación y comparación experimental

En este sprint se preparará y ejecutará la evaluación comparativa entre el *baseline* y la variante mejorada. Para ello, se terminará de construir el conjunto de casos de prueba, se definirán las rúbricas de evaluación manual y se recogerán las métricas seleccionadas para el análisis.

La comparación se centrará en dimensiones como la calidad de la recuperación, la fundamentación de las respuestas, la utilidad de los borradores generados y el coste temporal del sistema. Esta fase permitirá disponer de resultados cuantitativos con los que analizar el impacto real de la mejora propuesta.

3.6.7. Sprint 7: Revisión final, análisis de resultados y redacción de la memoria

La última fase del trabajo se dedicará a consolidar los resultados obtenidos, interpretar los datos recogidos durante la evaluación y redactar la memoria final del TFM. También se revisará el grado de cumplimiento de los objetivos planteados y se identificarán las principales limitaciones del sistema, así como posibles líneas de mejora o ampliación futura. Esta etapa incluirá además la revisión formal de la memoria.



Figura 2: Diagrama de Gantt con la planificación temporal del trabajo.

3.7. Recursos y costes

En esta sección se describen los recursos necesarios y se realiza una estimación orientativa de sus costes. Dado que se trata de un proyecto académico desarrollado de forma individual, la mayor parte del esfuerzo recae en la estudiante, mientras que la dirección del TFM asume funciones de supervisión, revisión y seguimiento.

3.7.1. Recursos humanos

Los recursos humanos considerados para el desarrollo del proyecto son los siguientes:

- **Dirección del TFM:** responsable de orientar el trabajo, revisar su evolución, validar las decisiones principales y supervisar la coherencia metodológica y académica del proyecto.
- **Estudiante:** responsable del análisis del problema, diseño de la arquitectura, implementación del prototipo, preparación del conjunto de evaluación, ejecución de los experimentos, análisis de resultados y redacción de la memoria final.

Aunque el proyecto es realizado por una única persona, desde el punto de vista de la estimación puede considerarse que la estudiante asume varias funciones de forma simultánea, entre ellas las de analista, desarrolladora e investigadora aplicada.

3.7.2. Recursos técnicos

Para el desarrollo del TFM se requerirán los siguientes recursos técnicos:

- Ordenador personal para desarrollo, pruebas y redacción de la memoria.
- Entorno de programación y herramientas de desarrollo para la implementación del prototipo.
- Herramientas de edición y documentación para la elaboración de diagramas, esquemas y memoria.
- Servicios o librerías necesarios para la implementación del sistema RAG, la integración del modelo generativo y el procesamiento documental.
- Herramientas de evaluación y análisis de resultados.

En términos generales, se priorizará el uso de herramientas gratuitas, académicas o ya disponibles en el entorno de trabajo, por lo que no se prevé un coste material elevado asociado al proyecto. En caso de utilizar servicios externos de pago, estos quedarán limitados a un uso puntual o experimental.

3.7.3. Estimación de dedicación

La siguiente tabla recoge una estimación orientativa de las horas de trabajo necesarias para desarrollar el proyecto:

Rol	Horas proyecto	Horas memoria	Total horas
Estudiante	300	120	420
Dirección del TFM	20	10	30

Tabla 1: Estimación de horas dedicadas al proyecto.

La dedicación principal corresponde a la estudiante, ya que asume tanto el desarrollo técnico del prototipo como la parte experimental y documental del trabajo. En cambio, la dirección del TFM participa de manera periódica mediante reuniones de seguimiento, revisión de avances y supervisión de la memoria.

3.7.4. Estimación de costes

A efectos de estimación, se asigna un coste por hora orientativo a cada uno de los roles considerados. Estos valores no deben interpretarse como costes contractuales reales, sino como una aproximación académica al esfuerzo invertido en el proyecto.

Rol	Total horas	Coste/hora (€)	Coste total (€)
Estudiante	420	25	10.500
Dirección del TFM	30	35	1.050
Total	450	–	11.550

Tabla 2: Estimación orientativa de costes humanos del proyecto.

Por tanto, el coste total estimado del proyecto, considerando únicamente los recursos humanos, sería de 11.550€. Esta cifra representa una valoración aproximada del esfuerzo necesario para desarrollar el asistente, implementar la mejora experimental propuesta y documentar adecuadamente el trabajo realizado.

Capítulo 4

Diseño e implementación de la propuesta

En este capítulo se describe el diseño técnico del asistente institucional propuesto, así como las principales decisiones adoptadas durante su implementación. Para ello, se presenta la arquitectura general del sistema, el flujo de ejecución del asistente, el papel del agente orquestador y las herramientas disponibles. Finalmente, se exponen las variantes implementadas y las tecnologías utilizadas para el desarrollo del prototipo.

4.1. Arquitectura general del sistema

La arquitectura general del sistema se organiza en torno a dos fases principales: una fase offline de preparación del corpus y una fase online de ejecución del asistente. Esta separación permite distinguir claramente entre las tareas necesarias para construir la base documental y las tareas que se realizan en tiempo de consulta.

La fase offline comprende la obtención, normalización, segmentación e indexación de los documentos que forman parte del corpus institucional. En esta etapa, las fuentes documentales se recopilan a partir de páginas y documentos públicos, se transforman a un formato homogéneo, se dividen en fragmentos o *chunks* y se enriquecen con metadatos. Posteriormente, dichos fragmentos se representan mediante embeddings y se almacenan en una base documental preparada para su recuperación posterior.

La fase online comienza cuando el usuario introduce una consulta en la interfaz conversacional. A partir de esa entrada, el agente orquestador interpreta la petición y decide qué capacidades del sistema deben activarse. En consultas informativas, el flujo principal consiste en recuperar evidencia documental relevante y generar una respuesta fundamentada. En

cambio, cuando la petición tiene una intención accionable, el orquestador puede activar una herramienta específica, como la generación de un borrador de correo.

La Figura 1 muestra una visión global de esta arquitectura. En ella se observa cómo la base documental alimenta la herramienta de recuperación, cómo el orquestador coordina las capacidades disponibles y cómo la capa de observabilidad permite registrar trazas del comportamiento del sistema durante la ejecución.

4.2. Flujo de ejecución del asistente

El flujo de ejecución del asistente parte de una consulta formulada en lenguaje natural por el usuario. La entrada se recibe a través de la interfaz conversacional y se envía al núcleo del sistema, donde el orquestador determina el tipo de tarea que debe resolverse. Esta decisión condiciona el resto del proceso, ya que no todas las consultas requieren el mismo tratamiento.

En el caso de preguntas informativas simples, el sistema sigue un flujo RAG directo. Primero, la consulta se transforma en una representación adecuada para el recuperador. Después, se buscan los fragmentos documentales más relevantes dentro de la base documental. Finalmente, el modelo de lenguaje genera una respuesta utilizando como contexto la evidencia recuperada.

Cuando la consulta requiere combinar información de varias fuentes, el sistema puede descomponer la pregunta en subconsultas o recuperar un conjunto más amplio de fragmentos candidatos. Este comportamiento resulta útil en preguntas comparativas o en consultas que no pueden resolverse a partir de un único documento. En estos casos, el objetivo no es únicamente recuperar un fragmento aislado, sino reunir evidencia suficiente para elaborar una respuesta más completa y estructurada.

En las preguntas no respondibles, el sistema debe evitar generar una respuesta no fundamentada. Por ello, si la evidencia recuperada no resulta suficiente o no guarda relación clara con la consulta, el flujo debe favorecer la abstención o una respuesta prudente que indique la falta de información disponible en el corpus. Este comportamiento es especialmente importante en un asistente institucional, donde proporcionar información incorrecta podría afectar negativamente a la confianza del usuario.

Por último, cuando la consulta expresa una intención accionable, como solicitar ayuda para redactar un correo formal, el orquestador activa la herramienta correspondiente. En esta versión del prototipo, dicha acción se limita a la generación de un borrador, manteniendo siempre una separación explícita entre proponer una acción y ejecutarla realmente.

4.3. Papel del orquestador

El agente orquestador actúa como componente de coordinación dentro del sistema. Su función principal consiste en interpretar la consulta del usuario, decidir qué flujo de ejecución resulta más adecuado y activar las herramientas necesarias para resolver la tarea. De este modo, el orquestador no sustituye al componente de recuperación ni al modelo de lenguaje, sino que organiza su uso dentro de una arquitectura controlada.

Desde una perspectiva agéntica, cada consulta se trata como una tarea que puede requerir distintos pasos. Algunas preguntas pueden resolverse mediante una recuperación documental directa, mientras que otras requieren comparar información, dividir la consulta en partes o activar una herramienta accionable. El orquestador permite introducir esta lógica de decisión sin acoplarla directamente al recuperador ni al modelo generativo.

En el prototipo, el orquestador contempla principalmente cinco tipos de comportamiento. En primer lugar, puede delegar una consulta simple al flujo RAG determinista. En segundo lugar, puede tratar consultas multi-documento mediante una estrategia más estructurada. En tercer lugar, puede organizar respuestas comparativas diferenciando entidades, criterios o condiciones. En cuarto lugar, puede abstenerse ante consultas no respondibles cuando no existe evidencia suficiente en el corpus. Finalmente, puede activar la herramienta de generación de borradores cuando detecta una petición orientada a acción.

Esta separación aporta trazabilidad y control al sistema. En lugar de generar siempre una respuesta del mismo modo, el asistente puede adaptar el flujo a la naturaleza de la consulta. Además, permite evaluar de forma independiente el impacto del orquestador sobre dimensiones como la abstención, la selección de herramientas y la estructuración de respuestas.

Para reforzar este control, el orquestador define un conjunto explícito de acciones permitidas que delimitan el ámbito de actuación del asistente. En concreto, las acciones válidas son las siguientes: respuesta informativa sobre un único documento (`answer_with_rag`), respuesta informativa que combina evidencia de múltiples fuentes (`multi_step_rag`), respuesta comparativa entre entidades o documentos (`comparative_rag`), generación supervisada de un borrador de correo electrónico (`draft_email`) y abstención explícita cuando la consulta no puede responderse con la evidencia disponible en el corpus (`refuse`). Cualquier consulta cuya intención quede fuera de este conjunto predefinido es rechazada antes de activar herramientas o iniciar la generación, convirtiendo al orquestador en una primera capa de *guardrails* del sistema. Este mecanismo evita que el asistente procese peticiones fuera del dominio definido, como solicitudes ajenas al ámbito institucional, peticiones que impliquen acciones no supervisadas o consultas que no puedan fundamentarse en el corpus documental disponible.

4.4. Herramientas disponibles

El sistema incorpora herramientas especializadas que pueden ser utilizadas por el orquestador durante la ejecución. En el contexto de este trabajo, una herramienta se entiende como un componente externo al modelo de lenguaje que proporciona una capacidad concreta, como recuperar información documental o generar una salida accionable.

La herramienta principal es la recuperación documental. Esta herramienta permite consultar la base documental construida previamente y devolver los fragmentos más relevantes para la pregunta del usuario. Su papel es fundamental en el sistema, ya que proporciona la evidencia necesaria para que las respuestas estén fundamentadas en información del corpus y no dependan únicamente del conocimiento paramétrico del modelo de lenguaje.

La segunda herramienta es la generación de borradores de correo. Esta funcionalidad permite transformar una consulta del usuario o una respuesta informativa previa en un texto formal orientado a una actuación posterior. En esta versión del prototipo, la herramienta se mantiene en modo simulado o *mockeado*: el sistema genera el borrador, pero no envía ningún mensaje real. Esta decisión permite validar el flujo conversacional y la utilidad de la salida generada sin introducir riesgos asociados al envío automático de correos.

El uso de herramientas se plantea de forma supervisada. El asistente puede proponer una acción, preparar un borrador o estructurar una respuesta accionable, pero la ejecución efectiva queda bajo control del usuario. Este enfoque resulta adecuado para un entorno institucional, donde es importante mantener trazabilidad, evitar acciones no deseadas y conservar la intervención humana en decisiones sensibles.

4.5. Variantes implementadas

Con el fin de evaluar el impacto de los distintos componentes del sistema, se implementaron varias variantes del prototipo. Estas variantes permiten comparar una configuración inicial sencilla con versiones que incorporan mejoras específicas en la recuperación documental o en el control del flujo mediante orquestación.

La primera variante corresponde al *baseline* determinista. Esta configuración sigue un flujo RAG directo: recibe la consulta del usuario, recupera fragmentos mediante búsqueda semántica y genera una respuesta a partir del contexto obtenido. Su objetivo es servir como punto de referencia para analizar el efecto de las mejoras posteriores.

La segunda variante introduce una mejora en el componente de recuperación de información. En este caso, la búsqueda semántica se complementa con recuperación léxica mediante

BM25 y con una etapa posterior de *reranking*. El objetivo de esta variante es comprobar si una recuperación más robusta permite situar la evidencia relevante en posiciones más altas del ranking y, por tanto, mejorar la calidad del contexto proporcionado al modelo de lenguaje.

La tercera variante incorpora el agente orquestador. En esta configuración, el sistema no aplica siempre el mismo flujo de forma determinista, sino que clasifica la consulta y decide cómo resolverla. Esta variante permite analizar el valor del orquestador en tareas como la abstención ante preguntas no respondibles, la identificación de solicitudes accionables y la activación de herramientas.

Una mejora relevante incorporada en la lógica del orquestador es la capacidad de gestionar consultas ambiguas o poco concretas. Cuando el sistema detecta que la pregunta del usuario carece de suficiente especificidad, no se limita a solicitar una aclaración, sino que primero genera una respuesta general basada en la evidencia disponible y, a continuación, plantea una pregunta de clarificación. De este modo, el usuario recibe información útil de inmediato y se fomenta un diálogo iterativo que permite afinar la respuesta en pasos sucesivos. Esta funcionalidad mejora la experiencia conversacional, ya que combina la entrega de valor inmediato con la capacidad de guiar al usuario hacia consultas más precisas, optimizando así la utilidad del asistente en escenarios reales donde las preguntas iniciales suelen ser vagas o incompletas.

Estas variantes se implementan de forma configurable, permitiendo activar o desactivar componentes como la recuperación híbrida, el *reranking* o el orquestador. Esta decisión facilita la comparación experimental posterior, ya que cada configuración puede ejecutarse sobre el mismo corpus y el mismo conjunto de consultas de evaluación.

Una vez descrito el diseño general del sistema, las siguientes secciones presentan las principales tecnologías utilizadas para implementar el prototipo. La selección se ha realizado atendiendo a criterios de modularidad, facilidad de integración, bajo coste, posibilidad de ejecución local y adecuación al desarrollo de sistemas basados en recuperación de información y modelos de lenguaje.

4.6. Lenguaje y entorno de desarrollo y ejecución

4.6.1. Python

El lenguaje principal del proyecto es Python. Esta elección se debe a su amplia adopción en el ámbito de la inteligencia artificial, el procesamiento del lenguaje natural y la computación científica. Su ecosistema incluye bibliotecas consolidadas para el tratamiento de datos, el aprendizaje automático y el uso de modelos de lenguaje, como NumPy, SciPy, scikit-learn o Trans-

formers (Harris y col., 2020; Virtanen y col., 2020; Pedregosa y col., 2011; Wolf y col., 2020). Esta disponibilidad de herramientas resulta especialmente adecuada para un prototipo basado en RAG, donde es necesario integrar procesamiento documental, generación de embeddings, recuperación semántica, reranking, evaluación y conexión con modelos generativos. Además, el uso de entornos virtuales y variables de configuración permite aislar dependencias y modificar parámetros del sistema sin alterar el código fuente, lo que facilita la reproducibilidad de las pruebas.

Frente a lenguajes como Java o Go, Python ofrece una curva de desarrollo más rápida y una integración más directa con las principales librerías utilizadas en proyectos de NLP y aprendizaje automático. Java y Go son alternativas robustas para servicios backend con altos requisitos de concurrencia, mantenibilidad o rendimiento. Sin embargo, en este trabajo el objetivo principal no es optimizar una infraestructura de producción, sino diseñar, implementar y evaluar distintas variantes de una arquitectura RAG.

También se ha valorado el uso de lenguajes de menor nivel, como C++, que podrían aportar ventajas en términos de rendimiento. No obstante, gran parte del coste computacional del sistema recae en la inferencia de modelos, la generación de embeddings y las consultas al almacén vectorial, componentes que ya suelen estar implementados mediante bibliotecas optimizadas o servicios externos. Por tanto, utilizar C++ para la lógica de orquestación o experimentación aumentaría la complejidad del desarrollo sin aportar una mejora proporcional al objetivo del prototipo.

Otro aspecto relevante es la reproducibilidad. Python permite aislar dependencias mediante entornos virtuales, de forma que cada entorno puede mantener sus propios paquetes y versiones sin interferir con la instalación global del sistema (Meyer, 2011). Esta característica facilita la ejecución controlada del prototipo, la comparación entre variantes experimentales y la futura replicación del trabajo.

4.6.2. Docker

Docker se utiliza como tecnología de contenedorización para ejecutar de forma aislada los servicios auxiliares del sistema. En el contexto de este proyecto, su uso permite levantar componentes como Qdrant, utilizado como base de datos vectorial, y Phoenix, empleado para la observabilidad del pipeline, sin instalarlos directamente sobre el sistema operativo anfitrión.

Para la definición y ejecución de estos servicios se emplea Docker Compose, que permite describir en un único fichero de configuración los servicios, redes y volúmenes necesarios para una aplicación multicontenedor (Docker Inc., 2026a). El uso de volúmenes permite conservar

información entre ejecuciones, evitando que datos como las colecciones vectoriales o las trazas generadas se pierdan al detener o recrear los contenedores (Docker Inc., 2026b). Esta característica resulta especialmente útil en un prototipo RAG, donde es necesario mantener el estado de servicios persistentes durante las fases de ingesta, evaluación y experimentación.

En comparación con una instalación manual de cada servicio, Docker reduce la dependencia de configuraciones específicas del equipo de desarrollo y simplifica la preparación del entorno.

4.7. Procesamiento documental

El procesamiento documental constituye una fase esencial del sistema, ya que condiciona la calidad de la información que posteriormente será recuperada por el modelo. En este proyecto, dicha fase incluye la obtención de documentos, su conversión a un formato homogéneo y su segmentación en fragmentos adecuados para la recuperación semántica.

4.7.1. Obtención del corpus y scraping web

El corpus del proyecto se construye exclusivamente a partir de información disponible en URLs públicas del dominio institucional. Para ello, se desarrolló un proceso de scraping propio basado en *httplib* y *BeautifulSoup*. La primera librería se utilizó para realizar peticiones HTTP de forma controlada, mientras que la segunda permitió analizar el contenido HTML, extraer el texto principal de las páginas y eliminar elementos no relevantes para la recuperación, como menús de navegación, pies de página, scripts o bloques puramente visuales.

El proceso de obtención del corpus no se planteó como un crawler generalista, sino como una recopilación acotada y reproducible de fuentes alineadas con el caso de uso del asistente institucional. Para ello, se partió de un conjunto inicial de URLs semilla seleccionadas manualmente por su relevancia para consultas habituales del dominio universitario, incluyendo información sobre trámites académicos, prácticas, becas, normativas, procedimientos administrativos y servicios para estudiantes.

Durante la recopilación se aplicaron criterios de inclusión y exclusión. Se incluyeron aquellas páginas que contenían información institucional textual, estable, verificable y potencialmente útil para responder preguntas del usuario. En cambio, se descartaron páginas que no superaban un umbral mínimo de 80 palabras de contenido útil tras la limpieza del HTML, páginas cuyo contenido quedaba compuesto mayoritariamente por elementos de navegación, banners o bloques visuales sin valor informativo, y páginas con contenido promocional o de actuali-

dad desconectado del dominio de consulta. Esta selección permitió construir un corpus más reducido, pero más controlado y coherente con los objetivos experimentales del trabajo.

El proceso de limpieza del HTML incluyó la eliminación sistemática de etiquetas de navegación (`nav`), cabeceras (`header`), pies de página (`footer`), formularios, barras laterales y bloques de JavaScript o CSS. Adicionalmente, se filtraron bloques HTML con atributos de clase o identificador asociados a patrones habituales de ruido en sitios web institucionales, como menús, *banners*, gestión de *cookies*, noticias, eventos, contenido social o elementos promocionales. Tras este proceso, únicamente se conservó el contenido principal de cada página, extraído preferentemente del elemento `main` o `article` cuando estuvo disponible. El cumplimiento del fichero `robots.txt` se verificó antes de cada descarga y se respetó un retardo mínimo entre peticiones para no sobrecargar los servidores.

Como resultado, de las 30 páginas descargadas correctamente, 4 fueron descartadas: tres no alcanzaron el umbral mínimo de palabras tras la limpieza y una presentó contenido mayoritariamente visual o estructural sin texto útil recuperable. Los 26 documentos restantes constituyen el corpus final empleado en la evaluación.

Además, el proceso se diseñó teniendo en cuenta criterios de reproducibilidad y trazabilidad. Cada documento recuperado conserva metadatos asociados a su fuente original, como la URL, el título, la categoría temática y el identificador del documento. Estos metadatos permiten posteriormente relacionar cada fragmento recuperado con su origen, analizar errores de recuperación y mostrar las fuentes utilizadas en las respuestas generadas por el asistente.

Como resultado de este proceso, el corpus final quedó formado por documentos institucionales públicos transformados a un formato textual homogéneo y preparados para su segmentación e indexación. Esta fase resulta especialmente relevante dentro del proyecto, ya que la calidad del corpus condiciona directamente la calidad de la recuperación documental y, por tanto, la fundamentación de las respuestas generadas por el sistema.

Elemento	Valor
URLs semilla consideradas	30
Páginas descargadas correctamente	30
Páginas descartadas durante la limpieza	4
Documentos finales incorporados al corpus	26
Formato final de almacenamiento	Markdown
Tamaño medio de documento	1557,92 palabras
Chunks generados	Pendiente de extraer
Tamaño medio de chunk	Pendiente de extraer

Tabla 3: Resumen del proceso de construcción del corpus documental.

4.7.2. Conversión y normalización de documentos

Una vez obtenidos los documentos, se aplica un proceso de normalización para reducir la heterogeneidad entre formatos. Las páginas HTML se convierten a Markdown mediante `markdownify`, lo que permite conservar una estructura textual sencilla y legible, eliminando al mismo tiempo elementos propios de la presentación web que no aportan información relevante para la recuperación (Anand, 2026).

En el caso de documentos PDF, se utiliza PyMuPDF, una librería orientada a la extracción, análisis y manipulación de documentos PDF desde Python (PyMuPDF, 2026). A diferencia de las fuentes HTML, los documentos PDF no se convierten a formato Markdown: su contenido textual se extrae directamente como texto plano, sin ninguna conversión de formato intermedia. Los ficheros de texto plano y Markdown se procesan directamente en su formato original. En definitiva, el formato Markdown se aplica exclusivamente a los documentos procedentes del *scraping* web, donde el proceso genera ficheros `.md` enriquecidos con metadatos en cabecera; para las fuentes PDF o de texto plano, el contenido se trata directamente sin conversión adicional. Este flujo permite transformar distintas fuentes en texto recuperable, facilitando las etapas posteriores de segmentación e indexación.

4.7.3. Segmentación del corpus y metadatos

Tras la normalización, los documentos se dividen en fragmentos o *chunks*. Esta segmentación es necesaria porque los modelos de embeddings y los modelos generativos trabajan con límites de contexto, y porque la recuperación resulta más precisa cuando se indexan unidades de información de tamaño controlado en lugar de documentos completos.

En este proyecto, la segmentación sigue una estrategia jerárquica orientada a preservar la coherencia semántica de los textos. En primer lugar, el documento se divide por límites de párrafo, identificados como bloques separados por una o más líneas en blanco. Si un párrafo supera el tamaño máximo de fragmento establecido (900 caracteres), se aplica una segunda división por oraciones mediante el reconocimiento de signos de puntuación terminal. En caso de que una oración individual resulte todavía demasiado larga, se recurre a una división por palabras como último recurso. Los fragmentos resultantes se construyen acumulando unidades hasta alcanzar el tamaño objetivo de 800 caracteres, con un margen entre 600 y 900 caracteres. Para evitar la pérdida de contexto entre fragmentos consecutivos, se aplica un solapamiento de 150 caracteres, de modo que el final de un fragmento se reutiliza como inicio del siguiente.

Esta estrategia jerárquica se prefiere frente a alternativas más simples por varias razones. La división por tamaño fijo de caracteres o *tokens*, pese a ser la opción más directa, produce

cortes arbitrarios que pueden interrumpir una oración, una enumeración o un concepto compuesto, lo que perjudica tanto la representación semántica del *embedding* como la coherencia del contexto proporcionado al modelo generativo. La división únicamente por oraciones genera fragmentos excesivamente pequeños que pueden carecer de contexto suficiente para que el recuperador comprenda el significado completo del pasaje. La segmentación únicamente por párrafos, sin control del tamaño máximo, puede producir fragmentos demasiado largos que superen la ventana de contexto útil del modelo de *embeddings* y diluyan la señal semántica en la representación vectorial. La estrategia adoptada busca un equilibrio entre estas restricciones, preservando la coherencia textual siempre que sea posible dentro de los límites de tamaño impuestos.

Además, cada fragmento conserva metadatos asociados a su documento de origen, lo que permite mantener la trazabilidad de las respuestas y recuperar la fuente original de la información utilizada por el sistema.

4.8. Almacenamiento vectorial

En los sistemas RAG, el almacenamiento vectorial permite guardar las representaciones numéricas de los fragmentos documentales y recuperarlos posteriormente mediante búsqueda por similitud semántica. Para esta función se ha seleccionado Qdrant, una base de datos vectorial orientada a la búsqueda semántica y al almacenamiento de vectores junto con información adicional o *payload* (Qdrant, 2026).

La elección de Qdrant se justifica por varias razones. En primer lugar, permite ejecutarse de forma local mediante Docker, lo que encaja con el enfoque de bajo coste y control del entorno planteado para el prototipo. En segundo lugar, ofrece persistencia de datos, filtrado por metadatos y una API de consulta que facilita su integración con el pipeline RAG. Estas características resultan especialmente útiles cuando se desea recuperar fragmentos no solo por similitud vectorial, sino también teniendo en cuenta información asociada a la fuente, el documento o la categoría del contenido.

Se valoraron otras alternativas ampliamente utilizadas en el ámbito de la recuperación vectorial. FAISS constituye una opción eficiente para búsqueda de similitud y clustering sobre vectores densos, pero se plantea principalmente como una librería y no como una base de datos vectorial completa con servicio persistente y API propia (Meta AI, 2026). Chroma, por su parte, ofrece una solución sencilla para almacenar embeddings, realizar búsquedas vectoriales y filtrar por metadatos (Chroma, 2026). Sin embargo, Qdrant se ajusta mejor a las necesidades del proyecto al proporcionar una separación más clara entre el servicio de almacenamiento

vectorial y el resto de componentes del sistema.

También se consideraron soluciones como Pinecone y Weaviate. Pinecone ofrece una base de datos vectorial gestionada orientada a aplicaciones de IA en producción (Pinecone, 2026), mientras que Weaviate proporciona una base de datos vectorial de código abierto con capacidades de búsqueda semántica e híbrida (Weaviate, 2026). No obstante, para este trabajo se prioriza una solución que pueda ejecutarse localmente, sin costes asociados por uso y con suficiente flexibilidad para las fases de ingesta, recuperación y evaluación.

4.9. Modelos de representación semántica

Los modelos de representación semántica permiten transformar consultas y fragmentos documentales en vectores numéricos, de forma que puedan compararse mediante medidas de similitud. Esta representación es uno de los componentes principales de una arquitectura RAG, ya que determina en gran medida qué información será recuperada y utilizada posteriormente por el modelo generativo.

4.9.1. BGE-M3

Para la generación de embeddings se ha seleccionado BGE-M3, integrado mediante la librería `SentenceTransformers`. Este modelo ha sido diseñado para tareas de recuperación multilingüe y destaca por su enfoque *multi-lingual*, *multi-functionality* y *multi-granularity*, lo que permite aplicarlo en distintos escenarios de búsqueda semántica (Chen y col., 2024a).

Resulta adecuado para un corpus en español y para posibles consultas formuladas en lenguaje natural y puede ejecutarse en un entorno local, lo que contribuye a mantener el control sobre los datos procesados. Por último, ofrece un equilibrio razonable entre calidad de recuperación y coste operativo, evitando la dependencia inicial de servicios externos de embeddings.

4.10. Recuperación de información

La recuperación de información es el proceso mediante el cual el sistema selecciona los fragmentos documentales más relevantes para una consulta. En el prototipo se contempla una recuperación densa basada en embeddings y una variante híbrida que combina similitud semántica con recuperación léxica.

4.10.1. Recuperación semántica

La recuperación semántica se realiza generando el embedding de la consulta y comparándolo con los vectores previamente almacenados en la base de datos vectorial. De esta forma, el sistema puede recuperar fragmentos relacionados por significado, aunque no compartan exactamente los mismos términos que la pregunta del usuario.

Este enfoque resulta especialmente útil en un asistente conversacional, ya que las consultas suelen formularse de manera natural y pueden diferir de la redacción exacta presente en los documentos del corpus.

4.10.2. Recuperación híbrida con BM25

Además de la recuperación densa, el sistema contempla una variante híbrida que combina la similitud vectorial con BM25, un método clásico de recuperación léxica basado en coincidencia de términos y ponderación estadística (Robertson y col., 2009). Esta combinación permite reforzar consultas en las que determinados términos concretos, como nombres de trámites, titulaciones o conceptos administrativos, son especialmente relevantes.

La recuperación híbrida permite equilibrar las ventajas de la búsqueda semántica con la precisión de los métodos léxicos, mejorando la robustez del sistema ante distintos tipos de preguntas.

4.11. Modelo de lenguaje

El modelo de lenguaje es el componente encargado de generar la respuesta final a partir de la pregunta del usuario y del contexto recuperado. En este proyecto, el pipeline se diseña de forma desacoplada respecto al proveedor concreto del modelo, de manera que sea posible utilizar distintas alternativas sin modificar la arquitectura general del sistema.

Durante el desarrollo del prototipo se prioriza el uso de proveedores compatibles con interfaces de tipo API, lo que facilita alternar entre modelos locales y servicios externos. En particular, herramientas como Ollama permiten ejecutar modelos localmente y ofrecen compatibilidad con interfaces similares a la API de OpenAI, lo que simplifica la integración con aplicaciones ya preparadas para este tipo de proveedores (Ollama, 2026).

Esta decisión permite mantener flexibilidad experimental: los modelos locales favorecen el control del entorno y la reducción de costes, mientras que las APIs externas pueden emplearse en fases concretas si se requiere una mayor calidad de generación. Así, el sistema no queda ligado a un único modelo ni a una única tecnología de inferencia.

4.12. Mejora de la recuperación mediante reranking

El reranking se introduce como una segunda etapa de recuperación. Tras obtener un conjunto inicial de fragmentos candidatos mediante recuperación semántica o híbrida, un modelo adicional vuelve a ordenar dichos fragmentos en función de su relevancia respecto a la consulta.

4.12.1. Modelo de reranking seleccionado

Para esta tarea se emplea un modelo de tipo `CrossEncoder` integrado mediante `SentenceTransformers`. A diferencia de los modelos bi-encoder utilizados para generar embeddings de forma independiente, un `CrossEncoder` evalúa conjuntamente el par formado por la consulta y cada fragmento candidato, lo que permite obtener una estimación más precisa de su relevancia (`SentenceTransformers`, 2026).

En el prototipo se contempla el uso de *BAAI/bge-reranker-v2-m3* como modelo principal de reranking, manteniendo la posibilidad de probar modelos más ligeros en la evaluación experimental. Esta separación permite comparar el impacto del reranking sobre la calidad de las respuestas sin modificar el resto del pipeline RAG.

4.13. Interfaz de usuario y observabilidad

La interfaz de usuario y la observabilidad son componentes de apoyo dentro del prototipo. Aunque no constituyen el núcleo del sistema RAG, resultan necesarias para facilitar la interacción con el asistente y analizar el comportamiento del pipeline durante las pruebas.

4.13.1. Interfaz conversacional con Streamlit

Para la interfaz conversacional se ha seleccionado Streamlit (`Streamlit`, 2026), un framework de código abierto orientado a la creación de aplicaciones interactivas en Python. Su principal ventaja en este proyecto es que permite construir una interfaz funcional con un esfuerzo reducido, sin introducir una capa de desarrollo frontend compleja.

Esta elección resulta adecuada porque el objetivo principal del trabajo no es el diseño de una interfaz avanzada, sino la implementación y evaluación de una arquitectura RAG con recuperación, reranking y orquestación. Por ello, Streamlit permite disponer de un entorno de prueba accesible para interactuar con el sistema, visualizar respuestas y comprobar el funcionamiento del pipeline de forma rápida.

Además, la interfaz se mantiene desacoplada del núcleo del sistema. Esta separación permite modificar o sustituir la capa visual en el futuro sin afectar a los componentes principales de ingesta, recuperación, generación u orquestación.

4.13.2. Trazabilidad y monitorización con Phoenix

Para la observabilidad del sistema se utiliza Phoenix (Arize AI, [2026](#)), una plataforma de código abierto orientada al análisis, depuración y evaluación de aplicaciones basadas en modelos de lenguaje. En el contexto de este proyecto, Phoenix permite inspeccionar las ejecuciones del pipeline y revisar elementos como las llamadas al modelo, los fragmentos recuperados, el uso de herramientas y los tiempos asociados a cada etapa.

La integración se apoya en OpenTelemetry (OpenTelemetry, [2026](#)), un framework abierto y neutral respecto al proveedor para generar, recopilar y exportar datos de telemetría, como trazas, métricas y registros. Esto permite instrumentar el prototipo sin acoplarlo a una solución cerrada de monitorización.

La observabilidad resulta especialmente relevante en un sistema RAG, ya que permite analizar por qué se ha generado una respuesta concreta, qué documentos han sido recuperados y en qué punto del pipeline pueden aparecer errores o degradaciones de calidad. De este modo, Phoenix no solo actúa como herramienta de depuración, sino también como apoyo para la evaluación experimental del sistema.

Capítulo 5

Resultados experimentales

Este capítulo presenta los resultados obtenidos durante la evaluación del asistente institucional desarrollado. El objetivo principal es analizar el comportamiento de las distintas variantes del sistema desde tres perspectivas complementarias: la calidad de la recuperación documental, la calidad de las respuestas generadas y la utilidad funcional del orquestador.

La evaluación se organiza en varios bloques. En primer lugar, se describe brevemente el corpus utilizado, el dataset de evaluación y las métricas empleadas. A continuación, se comparan los resultados de recuperación del baseline determinista frente a la variante con recuperación híbrida y reranking. Posteriormente, se analiza la calidad de las respuestas del flujo determinista y del orquestador. Finalmente, se discuten los resultados obtenidos, destacando las mejoras observadas y las principales limitaciones del proceso de evaluación.

5.1. Diseño de la evaluación

La evaluación experimental se diseñó con el propósito de comparar el comportamiento del sistema en diferentes configuraciones. Para ello, se utilizó un corpus documental construido a partir de información pública de UNIR y un conjunto de preguntas diseñado específicamente para cubrir distintos tipos de consulta: preguntas respondibles desde un único documento, preguntas que requieren combinar información de varias fuentes, preguntas comparativas y preguntas no respondibles a partir del corpus disponible.

5.1.1. Corpus y dataset de evaluación

La evaluación se realizó sobre un corpus documental construido a partir de información pública del dominio institucional. Tras el proceso de recopilación, limpieza, normalización y segmentación descrito en el capítulo anterior, el corpus final quedó formado por 26 documentos en

formato Markdown. Este formato se seleccionó porque permite conservar una estructura textual sencilla, facilitar la segmentación por bloques de contenido y mantener una representación homogénea de fuentes originalmente heterogéneas.

A partir de este corpus se construyó manualmente un dataset de evaluación compuesto por 30 preguntas representativas del dominio. El objetivo de este conjunto no es constituir un benchmark general de asistentes institucionales, sino proporcionar una validación controlada del prototipo y permitir la comparación de distintas variantes del sistema bajo las mismas condiciones experimentales.

Las preguntas se diseñaron buscando cubrir escenarios habituales de uso en un contexto universitario. En concreto, se incluyeron preguntas respondibles desde un único documento, preguntas que requieren combinar información procedente de varias fuentes, preguntas comparativas y preguntas no respondibles a partir del corpus disponible. Esta última categoría se incorporó de forma deliberada para evaluar la capacidad del sistema de abstenerse cuando no existe evidencia suficiente, un comportamiento especialmente importante en asistentes institucionales donde la generación de información no fundamentada puede afectar a la confianza del usuario.

Para las preguntas respondibles se anotó manualmente la fuente esperada, es decir, el documento o documentos en los que debía encontrarse la evidencia necesaria para responder. Además, en un subconjunto de preguntas se definieron hechos esperados, entendidos como unidades mínimas de información que una respuesta correcta debería incluir. Esta anotación permitió evaluar no solo si el sistema recuperaba la fuente adecuada, sino también si la respuesta generada incorporaba los elementos informativos relevantes.

El tamaño del dataset constituye una limitación de la evaluación, ya que 30 preguntas no permiten extraer conclusiones generalizables sobre cualquier asistente institucional. No obstante, se considera adecuado para una primera validación académica del prototipo, dado que permite comparar de forma controlada las variantes implementadas, analizar errores de recuperación y respuesta, y estudiar el comportamiento del orquestador ante distintos tipos de consulta. Por tanto, los resultados deben interpretarse como una evaluación inicial y controlada, no como una medición definitiva del rendimiento del sistema en producción.

Elemento	Valor
Documentos finales del corpus	26
URLs semilla consideradas	30
Páginas descargadas correctamente	30
Páginas descartadas durante la limpieza	4
Formato de almacenamiento	Markdown
Tamaño medio de documento	1557,92 palabras
Preguntas totales del dataset	30
Preguntas respondibles	25
Preguntas no respondibles	5
Preguntas con fuente esperada anotada	25
Preguntas con hechos esperados anotados	Pendiente de extraer

Tabla 4: Resumen del corpus y del dataset utilizados en la evaluación.

Estos valores reflejan un proceso de recopilación acotado y controlado. Aunque el número de documentos finales no es elevado, el corpus se construyó a partir de fuentes seleccionadas manualmente y revisadas durante la limpieza, priorizando la relevancia institucional, la trazabilidad de la fuente y la utilidad para el conjunto de preguntas de evaluación. Por tanto, el objetivo no es maximizar el volumen documental, sino disponer de una base suficientemente representativa y coherente para validar el comportamiento del prototipo bajo condiciones comparables.

Tipo de pregunta	Número de preguntas
Respondible desde un único documento	15
Respondible con combinación de documentos	6
Comparativa	4
No respondible	5

Tabla 5: Distribución de preguntas del dataset de evaluación.

5.1.2. Métricas empleadas

Para evaluar el sistema se utilizaron métricas diferentes en función del componente analizado. En la fase de recuperación documental se emplearon métricas habituales en sistemas de búsqueda de información:

- **Hit@1**: indica si la fuente esperada aparece en la primera posición de los resultados recuperados.
- **Hit@3**: indica si la fuente esperada aparece entre los tres primeros resultados recuperados.
- **MRR**: mide la posición media recíproca de la primera fuente relevante recuperada.

Para evaluar la calidad de las respuestas se utilizaron métricas orientadas al contenido generado:

- **Cobertura de hechos esperados:** proporción de hechos esperados presentes en la respuesta generada.
- **Tasa de abstención:** proporción de preguntas no respondibles en las que el sistema evita responder sin evidencia suficiente.
- **Exactitud en la detección de intención de correo:** capacidad del orquestador para identificar solicitudes que requieren activar la herramienta de generación de borradores.

5.1.3. Variantes evaluadas

La evaluación se centró en tres variantes principales del sistema, representativas de distintos niveles de complejidad. La primera variante corresponde al baseline determinista, basado en recuperación semántica simple. La segunda introduce recuperación híbrida y reranking para mejorar la ordenación de la evidencia recuperada. La tercera incorpora un orquestador encargado de clasificar la consulta, decidir el flujo de ejecución y activar herramientas cuando corresponde.

Variante	Recuperación	Reranking	Orquestador
A: Baseline determinista	Densa	No	No
B: Híbrida + reranking	Densa + léxica	Sí	No
C: Orquestador	Adaptativa	No	Sí

Tabla 6: Variantes evaluadas durante los experimentos.

La evaluación separa deliberadamente la mejora del componente de recuperación y la evaluación funcional del orquestador. De este modo, la variante B permite analizar el impacto de la recuperación híbrida y el *reranking* sobre la calidad de la evidencia recuperada, mientras que la variante C permite estudiar el efecto del orquestador sobre el control del flujo, la abstención ante preguntas no respondibles y la activación de herramientas. La combinación completa de ambas líneas en una variante adicional, basada en orquestador con recuperación híbrida y *reranking*, queda planteada como una extensión prevista para la siguiente iteración del trabajo.

5.2. Resultados de recuperación

En esta sección se comparan los resultados obtenidos por el baseline determinista y la variante con recuperación híbrida y reranking. La evaluación se realiza sobre las 25 preguntas del dataset que cuentan con una fuente esperada, excluyendo las preguntas no respondibles.

Variante	Hit@1	Δ	Hit@3	Δ	MRR	Δ
A: Baseline determinista	0.520	–	0.760	–	0.655	–
B: Híbrida + reranking	0.600	+0.080	0.800	+0.040	0.724	+0.069

Tabla 7: Comparativa de recuperación entre el baseline y la variante con recuperación híbrida y reranking.

Los resultados muestran una mejora consistente de la variante híbrida con reranking respecto al baseline determinista. En concreto, Hit@1 aumenta de 0.520 a 0.600, lo que indica que la fuente esperada aparece con mayor frecuencia en la primera posición. Asimismo, Hit@3 mejora de 0.760 a 0.800, reflejando una mayor presencia de evidencia relevante entre los primeros resultados recuperados.

La mejora también se observa en MRR, que pasa de 0.655 a 0.724. Este incremento sugiere que la evidencia relevante no solo aparece con mayor frecuencia en el conjunto de resultados, sino que además tiende a situarse en posiciones más altas dentro del ranking. Por tanto, la incorporación de recuperación híbrida y reranking contribuye positivamente a la calidad del componente de recuperación.

5.3. Resultados de calidad de respuesta

Además de evaluar la recuperación documental, se analizaron las respuestas generadas por el sistema. Esta evaluación permite observar si las mejoras introducidas en la arquitectura se traducen en respuestas más completas, más seguras o más controladas.

La Tabla 8 compara el comportamiento del flujo determinista y del orquestador en tres dimensiones: cobertura de hechos esperados, abstención ante preguntas no respondibles y detección de solicitudes de correo.

Métrica	A: Determinista	C: Orquestador
Cobertura de hechos esperados	0.438	0.312
Tasa de abstención en preguntas no respondibles	0.000	1.000
Exactitud en detección de intención de correo	0.000	0.833

Tabla 8: Comparativa de calidad de respuesta entre el flujo determinista y el orquestador.

Los resultados muestran un comportamiento diferenciado entre ambas variantes. El flujo determinista obtiene una mayor cobertura de hechos esperados en preguntas respondibles, con un valor de 0.438 frente al 0.312 obtenido por el orquestador. Este resultado indica que, en esta configuración, el flujo determinista tiende a producir respuestas más alineadas con los hechos esperados definidos en el dataset.

Sin embargo, el orquestador mejora de forma clara el comportamiento ante preguntas no respondibles. Mientras que el flujo determinista no se abstiene en este tipo de preguntas, el orquestador alcanza una tasa de abstención de 1.000. Este resultado es especialmente relevante en un asistente institucional, ya que responder sin evidencia suficiente puede producir información incorrecta o poco fiable.

Además, el orquestador permite detectar solicitudes accionables, como la generación de borradores de correo. En esta dimensión alcanza una exactitud de 0.833, frente al 0.000 del flujo determinista, que no incorpora mecanismos de planificación ni activación de herramientas. Por tanto, aunque el orquestador no mejora la cobertura factual en esta evaluación, sí aporta capacidades relevantes de control, abstención y uso de herramientas.

5.4. Evaluación funcional del orquestador

La evaluación del orquestador no se limita únicamente a métricas de respuesta, ya que su objetivo principal no es sustituir al recuperador, sino coordinar el flujo de ejecución del asistente. Por ello, se realizaron pruebas funcionales orientadas a comprobar si el sistema identifica correctamente el tipo de consulta y selecciona la acción adecuada.

La Tabla 9 resume el comportamiento observado en consultas representativas de los principales escenarios contemplados.

Tipo de consulta	Comportamiento esperado	Resultado observado
single_doc	Respuesta RAG directa	Correcto
multi_doc	Descomposición en subconsultas	Correcto
comparative	Comparación estructurada por entidades	Correcto
unanswerable	Abstención ante falta de evidencia	Correcto
Solicitud de correo	Generación de borrador	Correcto

Tabla 9: Validación funcional del comportamiento del orquestador.

Estos resultados muestran que el orquestador cumple su función principal como componente de control. En consultas simples, permite delegar la respuesta al flujo RAG directo. En consultas que requieren varios pasos, genera una planificación más estructurada. En preguntas comparativas, organiza la respuesta por entidades y diferencias principales. Finalmente, ante preguntas no respondibles o solicitudes accionables, activa comportamientos específicos que no están disponibles en el baseline determinista.

5.5. Discusión de resultados

Los resultados obtenidos permiten extraer varias conclusiones relevantes sobre el comportamiento del sistema.

En primer lugar, la incorporación de recuperación híbrida y reranking mejora el rendimiento del sistema en las métricas de recuperación. La mejora en Hit@1 y MRR indica que la evidencia relevante aparece mejor posicionada, lo que resulta especialmente importante en sistemas RAG, donde la calidad de la generación depende directamente del contexto recuperado.

En segundo lugar, el orquestador no mejora la cobertura de hechos esperados respecto al flujo determinista en la configuración evaluada. Este resultado puede explicarse por dos factores. Por un lado, el orquestador adopta un comportamiento más conservador, especialmente cuando la evidencia recuperada no es suficientemente clara. Por otro lado, la métrica de cobertura de hechos esperados depende de la coincidencia entre la respuesta generada y un conjunto concreto de hechos definidos previamente, por lo que puede penalizar respuestas correctas pero formuladas de manera más general o prudente.

En tercer lugar, el orquestador sí aporta mejoras claras en dimensiones que no quedan reflejadas únicamente mediante métricas clásicas de recuperación. Su capacidad para abstenerse ante preguntas no respondibles reduce el riesgo de generar respuestas no fundamentadas. Asimismo, la detección de solicitudes de correo demuestra que la arquitectura puede extenderse hacia funcionalidades accionables, manteniendo un flujo controlado y trazable.

En conjunto, los resultados sugieren que las mejoras sobre el recuperador tienen un impacto directo en la calidad de la evidencia recuperada, mientras que el orquestador aporta valor principalmente en términos de control del flujo, seguridad de la respuesta y extensibilidad funcional. Por tanto, ambas líneas de mejora son complementarias: el reranking contribuye a recuperar mejor información, mientras que el orquestador permite decidir cómo utilizarla.

Por tanto, los resultados deben interpretarse desde la contribución global de la arquitectura: la mejora del retrieval incrementa la calidad de la evidencia disponible, mientras que la orquestación permite controlar cómo se utiliza dicha evidencia y cuándo debe activarse una capacidad accionable. La aportación principal no reside en un único componente aislado, sino en la integración modular y trazable de estos elementos dentro de un asistente institucional supervisado.

5.6. Limitaciones de la evaluación

La evaluación realizada presenta algunas limitaciones que deben tenerse en cuenta al interpretar los resultados. En primer lugar, el tamaño del corpus y del dataset es reducido, por lo que los valores obtenidos deben entenderse como una aproximación inicial al comportamiento del sistema y no como una medición definitiva.

En segundo lugar, la evaluación de calidad de respuesta se basa en un subconjunto de preguntas con hechos esperados definidos manualmente. Este enfoque permite realizar una comparación semiautomatizada, pero puede penalizar respuestas que sean parcialmente correctas o que expresen la información con una formulación distinta a la esperada.

En tercer lugar, el modelo generativo utilizado en la evaluación es ligero, lo que puede afectar a la calidad final de las respuestas. En consecuencia, parte de los errores observados pueden deberse no solo al diseño del pipeline RAG, sino también a las limitaciones del modelo de lenguaje empleado.

Finalmente, la evaluación funcional del orquestador permite comprobar que los flujos principales se comportan como se esperaba, pero sería conveniente ampliarla en trabajos futuros con un conjunto mayor de consultas y métricas específicas de routing, planificación y uso de herramientas.

5.7. Síntesis de resultados

A modo de síntesis, los experimentos realizados muestran que la variante con recuperación híbrida y reranking mejora los resultados de recuperación respecto al baseline determinista. Esta mejora se observa tanto en la presencia de la fuente correcta entre los primeros resultados como en su posición media dentro del ranking.

Por otro lado, la comparación entre el flujo determinista y el orquestador muestra que el segundo no incrementa la cobertura de hechos esperados en esta configuración, pero sí mejora de forma notable la abstención ante preguntas no respondibles y permite incorporar capacidades accionables. Estos resultados respaldan la utilidad de una arquitectura modular, en la que las mejoras de recuperación y las capacidades de orquestación cumplen funciones distintas pero complementarias dentro del asistente institucional.

Bibliografía

- Robertson, Stephen y Hugo Zaragoza (2009). «The Probabilistic Relevance Framework: BM25 and Beyond». En: *Foundations and Trends in Information Retrieval* 3.4, págs. 333-389. doi: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- Meyer, Carl (2011). *PEP 405 – Python Virtual Environments*. Accessed: 2026-05-19. url: <https://peps.python.org/pep-0405/>.
- Pedregosa, Fabian y col. (2011). «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* 12.85, págs. 2825-2830. url: <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- Harris, Charles R. y col. (2020). «Array programming with NumPy». En: *Nature* 585, págs. 357-362. doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- Karpukhin, Vladimir y col. (2020). «Dense Passage Retrieval for Open-Domain Question Answering». En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, págs. 6769-6781. doi: [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550).
- Lewis, Patrick y col. (2020). «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks». En: *Advances in Neural Information Processing Systems*.
- Nogueira, Rodrigo y col. (2020a). «Document Ranking with a Pretrained Sequence-to-Sequence Model». En: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, págs. 708-718. doi: [10.18653/v1/2020.findings-emnlp.63](https://doi.org/10.18653/v1/2020.findings-emnlp.63). url: <https://aclanthology.org/2020.findings-emnlp.63/>.
- (2020b). «Document Ranking with a Pretrained Sequence-to-Sequence Model». En: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, págs. 708-718. doi: [10.18653/v1/2020.findings-emnlp.63](https://doi.org/10.18653/v1/2020.findings-emnlp.63).
- Virtanen, Pauli y col. (2020). «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17.3, págs. 261-272. doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- Wolf, Thomas y col. (2020). «Transformers: State-of-the-Art Natural Language Processing». En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*:

- System Demonstrations*. Association for Computational Linguistics, págs. 38-45. doi: [10.18653/v1/2020.emnlp-demos.6](https://doi.org/10.18653/v1/2020.emnlp-demos.6). url: <https://aclanthology.org/2020.emnlp-demos.6/>.
- Qu, Yingqi y col. (2021). «RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering». En: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, págs. 5835-5847. doi: [10.18653/v1/2021.naacl-main.466](https://doi.org/10.18653/v1/2021.naacl-main.466).
- Li, Minghao y col. (2023). «API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs». En: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, págs. 3102-3116. doi: [10.18653/v1/2023.emnlp-main.187](https://doi.org/10.18653/v1/2023.emnlp-main.187).
- Schick, Timo y col. (2023). «Toolformer: Language Models Can Teach Themselves to Use Tools». En: *Advances in Neural Information Processing Systems*.
- Yao, Shunyu y col. (2023). «ReAct: Synergizing Reasoning and Acting in Language Models». En: *The Eleventh International Conference on Learning Representations*.
- Asai, Akari y col. (2024). «Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection». En: *The Twelfth International Conference on Learning Representations*.
- Chen, Jianlv y col. (2024a). «BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation». En: *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, págs. 8753-8768. url: <https://aclanthology.org/2024.findings-acl.137/>.
- Chen, Weize y col. (2024b). «AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors». En: *The Twelfth International Conference on Learning Representations*.
- Hong, Sirui y col. (2024). «MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework». En: *The Twelfth International Conference on Learning Representations*.
- Jeong, Soyeong y col. (2024). «Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity». En: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, págs. 7036-7050. doi: [10.18653/v1/2024.naacl-long.389](https://doi.org/10.18653/v1/2024.naacl-long.389).
- Patil, Shishir G. y col. (2024). «Gorilla: Large Language Model Connected with Massive APIs». En: *Advances in Neural Information Processing Systems*. doi: [10.52202/079017-4020](https://doi.org/10.52202/079017-4020).

- Qin, Yujia y col. (2024). «ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs». En: *The Twelfth International Conference on Learning Representations*.
- Saad-Falcon, Jon y col. (2024). «ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems». En: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, págs. 338-354. doi: [10.18653/v1/2024.naacl-long.20](https://doi.org/10.18653/v1/2024.naacl-long.20).
- Shi, Weijia y col. (2024). «REPLUG: Retrieval-Augmented Black-Box Language Models». En: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, págs. 8371-8384. doi: [10.18653/v1/2024.naacl-long.463](https://doi.org/10.18653/v1/2024.naacl-long.463).
- Wu, Qingyun y col. (2024). «AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations». En: *Proceedings of the First Conference on Language Modeling*.
- Zeng, Shenglai y col. (2024). «The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)». En: *Findings of the Association for Computational Linguistics: ACL 2024*, págs. 4505-4524. doi: [10.18653/v1/2024.findings-acl.267](https://doi.org/10.18653/v1/2024.findings-acl.267).
- Dang, Yufan y col. (2025). «Multi-Agent Collaboration via Evolving Orchestration». En: *Advances in Neural Information Processing Systems*.
- Kalra, Rishi y col. (abr. de 2025). «HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications». En: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. Albuquerque, New Mexico: Association for Computational Linguistics, págs. 1036-1054. doi: [10.18653/v1/2025.naacl-industry.79](https://doi.org/10.18653/v1/2025.naacl-industry.79).
- Liu, Hanchao y col. (abr. de 2025). «WorkTeam: Constructing Workflows from Natural Language with Multi-Agents». En: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. Albuquerque, New Mexico: Association for Computational Linguistics, págs. 20-35. doi: [10.18653/v1/2025.naacl-industry.3](https://doi.org/10.18653/v1/2025.naacl-industry.3).
- Nguyen, Long S. T. y col. (dic. de 2025). «When in Doubt, Ask First: A Unified Retrieval Agent-Based System for Ambiguous and Unanswerable Question Answering». En: *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*.

- Mumbai, India: The Asian Federation of Natural Language Processing y The Association for Computational Linguistics, págs. 452-472. doi: [10.18653/v1/2025.findings-ijcnlp.27](https://doi.org/10.18653/v1/2025.findings-ijcnlp.27).
- Trienes, Jan y col. (2025). «Marcel: A Lightweight and Open-Source Conversational Agent for University Student Support». En: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Suzhou, China: Association for Computational Linguistics, págs. 181-195. url: <https://aclanthology.org/2025.emnlp-demos.13/>.
- Zhang, Xuan y col. (nov. de 2025). «AskToAct: Enhancing LLMs Tool Use via Self-Correcting Clarification». En: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, págs. 13484-13511 doi: [10.18653/v1/2025.emnlp-main.682](https://doi.org/10.18653/v1/2025.emnlp-main.682).
- Anand, Matthew (2026). *python-markdownify: Convert HTML to Markdown*. Accessed: 2026-05-19. url: <https://github.com/matthewwithanm/python-markdownify>.
- Arize AI (2026). *Phoenix Documentation*. Accessed: 2026-05-19. url: <https://arize.com/docs/phoenix>.
- Chroma (2026). *Chroma Documentation*. <https://docs.trychroma.com/>. Accessed: 2026-05-19.
- Docker Inc. (2026a). *Docker Compose documentation*. Accessed: 2026-05-19. url: <https://docs.docker.com/compose/>.
- (2026b). *Persisting container data*. Accessed: 2026-05-19. url: <https://docs.docker.com/get-started/docker-concepts/running-containers/persisting-container-data/>.
- Meta AI (2026). *FAISS Documentation*. <https://faiss.ai/>. Accessed: 2026-05-19.
- Ollama (2026). *OpenAI compatibility*. Accessed: 2026-05-19. url: <https://docs.ollama.com/api/openai-compatibility>.
- OpenTelemetry (2026). *OpenTelemetry Documentation*. Accessed: 2026-05-19. url: <https://opentelemetry.io/docs/>.
- Pinecone (2026). *Pinecone Documentation*. <https://docs.pinecone.io/>. Accessed: 2026-05-19.
- PyMuPDF (2026). *PyMuPDF documentation*. Accessed: 2026-05-19. url: <https://pymupdf.readthedocs.io/>.
- Qdrant (2026). *Qdrant Documentation*. <https://qdrant.tech/documentation/>. Accessed: 2026-05-19.
- Sentence Transformers (2026). *Cross-Encoders*. Accessed: 2026-05-19. url: https://www.sbert.net/examples/cross_encoder/applications/README.html.

Streamlit (2026). *Streamlit Documentation*. Accessed: 2026-05-19. url: <https://docs.streamlit.io/>.

Weaviate (2026). *Weaviate Documentation*. <https://weaviate.io/developers/weaviate>. Accessed: 2026-05-19.

Apéndices

Apéndice A

Fusce viverra lectus

Nam viverra, odio et vulputate ultricies, sem libero ornare nunc, nec suscipit urna sapien eu sapien. Nulla erat nulla, hendrerit non elit vitae, venenatis malesuada libero. Duis ornare sapien sed lacus condimentum rutrum. Donec feugiat erat id elit aliquam, a ultrices lectus mattis. In vitae nisl tortor. Proin pellentesque nec odio et posuere. Ut aliquet quam ac magna tincidunt ultrices. Nullam sit amet elementum leo. Pellentesque vitae mi dolor. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Suspendisse tincidunt mi arcu, vitae porttitor mauris elementum ut.

In vel condimentum diam. Fusce viverra lectus sed ultrices tristique. Sed euismod sem tincidunt porttitor mollis. Aenean venenatis sem eros, vel ultrices lectus euismod ac. Nam molestie ex vitae volutpat porta. Proin eu turpis ut ipsum rhoncus ultricies nec eu lectus. Cras malesuada fringilla rutrum. In posuere, lectus fermentum molestie mattis, metus ligula ullamcorper sapien, id elementum nulla odio eget odio. Aenean risus velit, pretium at egestas mattis, aliquet scelerisque massa. Vivamus in mi sed mauris mollis fringilla quis quis ante.